

第 8 章 人工神经网络及其应用

教材：

王万良 《人工智能导论》（第5版）

高等教育出版社，2020

第8章 人工神经网络及其应用

- 人工神经网络是对人脑或生物神经网络若干基本特性的抽象和模拟。为机器学习等许多问题的研究提供了一条新的思路，目前已经在模式识别、机器视觉、联想记忆、自动控制、信号处理、软测量、决策分析、智能计算、组合优化问题求解、数据挖掘等方面获得成功应用。
- 本章着重介绍最基本、最典型、应用最广泛的**BP**神经网络和**Hopfield**神经网络及其在模式识别、联想记忆、软测量、智能计算、组合优化问题求解等方面的应用。

第8章 人工神经网络及其应用

● 神经网络（neural networks, NN）

- **生物神经网络**(natural neural network, NNN): 由中枢神经系统（脑和脊髓）及周围神经系统（感觉神经、运动神经等）所构成的错综复杂的神经网络，其中最重要的是**脑神经系统**。
- **人工神经网络**(artificial neural networks, ANN): 模拟**人脑神经系统的**结构和功能，运用大量简单处理单元经广泛连接而组成的人工网络系统。

神经网络方法：**隐式**的
知识表示方法

第8章 人工神经网络及其应用

- 8.1 神经元与神经网络
- 8.2 BP神经网络及其学习算法
- 8.3 BP神经网络在模式识别中的应用
- 8.4 Hopfield神经网络及其改进
- 8.5 Hopfield神经网络的应用
- 8.6 卷积神经网络与深度学习
- 8.7 生成对抗网络及其应用

第8章 人工神经网络及其应用

- ✓ 8.1 神经元与神经网络
- 8.2 BP神经网络及其学习算法
- 8.3 BP神经网络在模式识别中的应用
- 8.4 Hopfield神经网络及其改进
- 8.5 Hopfield神经网络的应用
- 8.6 卷积神经网络与深度学习
- 8.7 生成对抗网络及其

8.1 神经元与神经网络

- 8.1.1 生物神经元的结构
- 8.1.2 神经元数学模型
- 8.1.3 神经网络结构与工作方式
- 8.1.4 神经网络的学习

8.1.1 生物神经元的结构

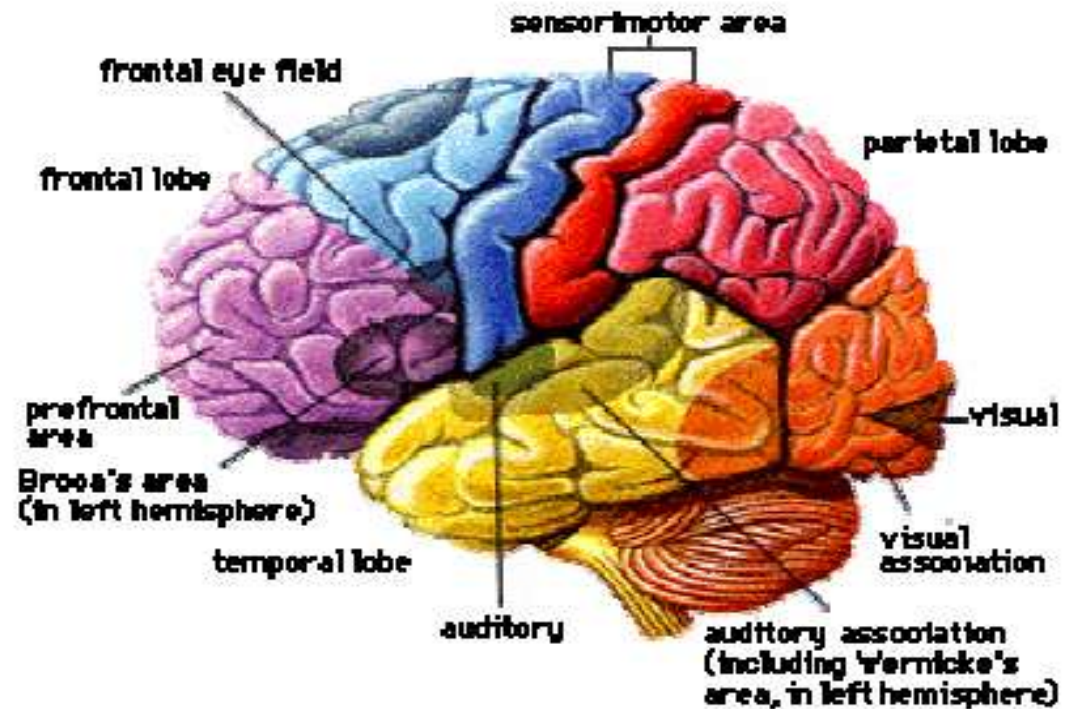
■ 人脑构造:

皮层 (cortex)

中脑 (midbrain)

脑干 (brainstem)

小脑 (cerebellum)

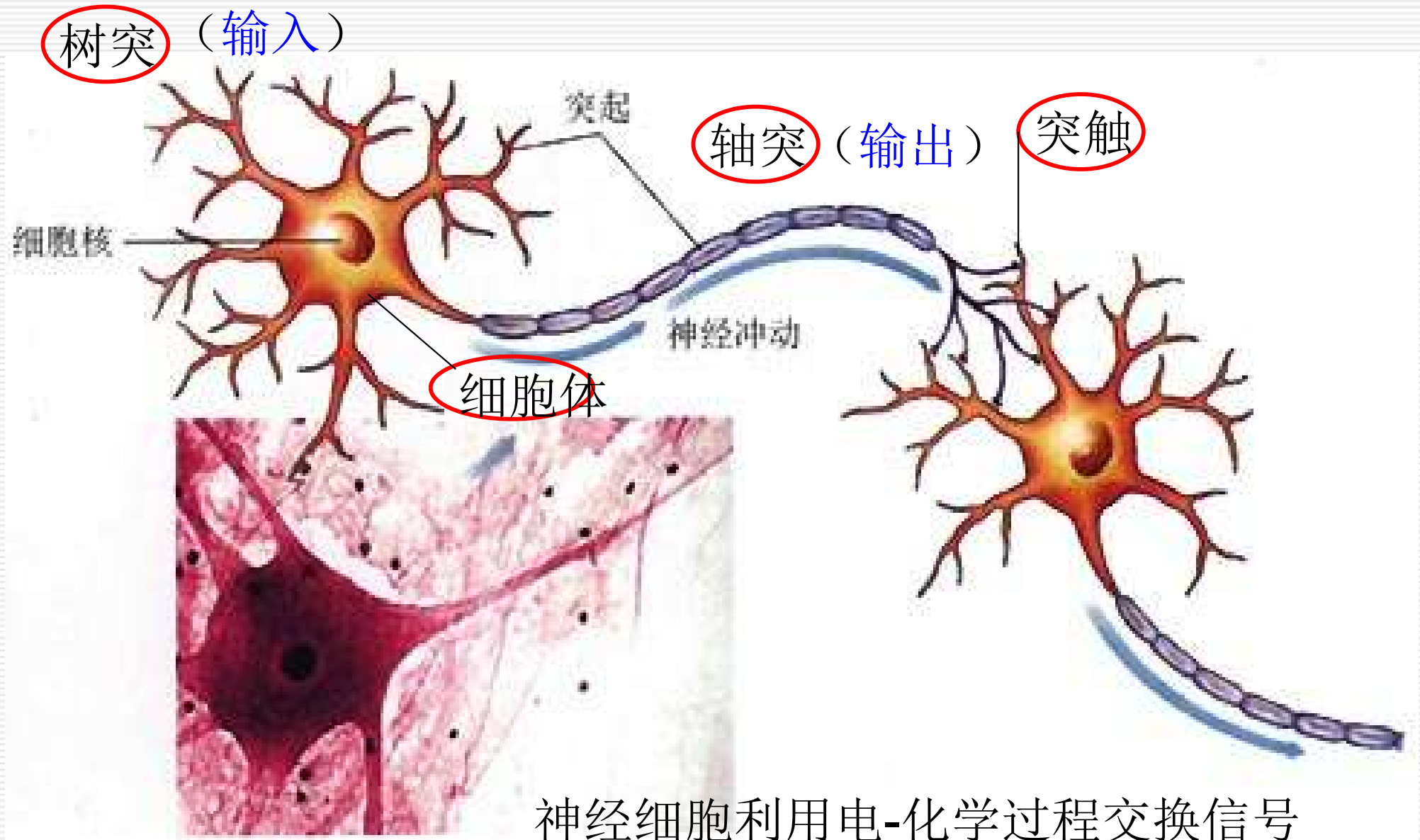


■ 人脑由一千多亿（ 10^{11} 亿— 10^{14} 亿）个神经细胞（神经元）交织在一起的网状结构组成，其中大脑皮层约140亿个神经元，小脑皮层约1000亿个神经元。

■ 神经元约有1000种类型，每个神经元大约与 10^3 — 10^4 个其他神经元相连接，形成极为错综复杂而又灵活多变的神经网络。

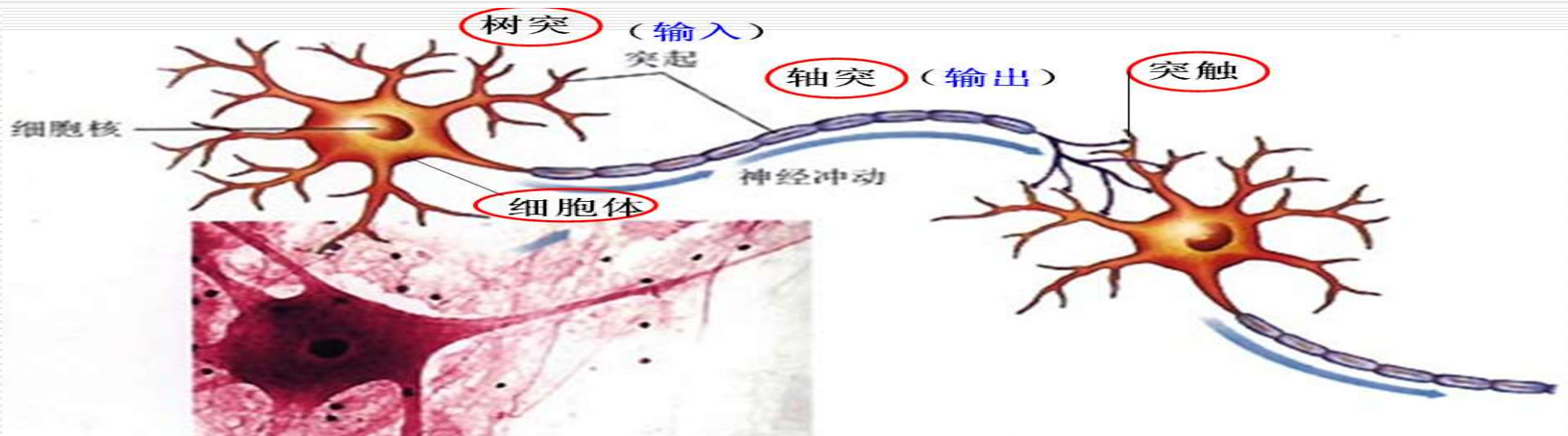
8.1.1 生物神经元的结构

□ 1. 生物神经元结构



8.1.1 生物神经元的结构

- 工作状态：
 - 兴奋状态：细胞膜电位 $>$ 动作电位的阈值 $\rightarrow$ 神经冲动
 - 抑制状态：细胞膜电位 $<$ 动作电位的阈值
- 学习与遗忘：由于神经元结构的可塑性，突触的传递作用可增强和减弱。



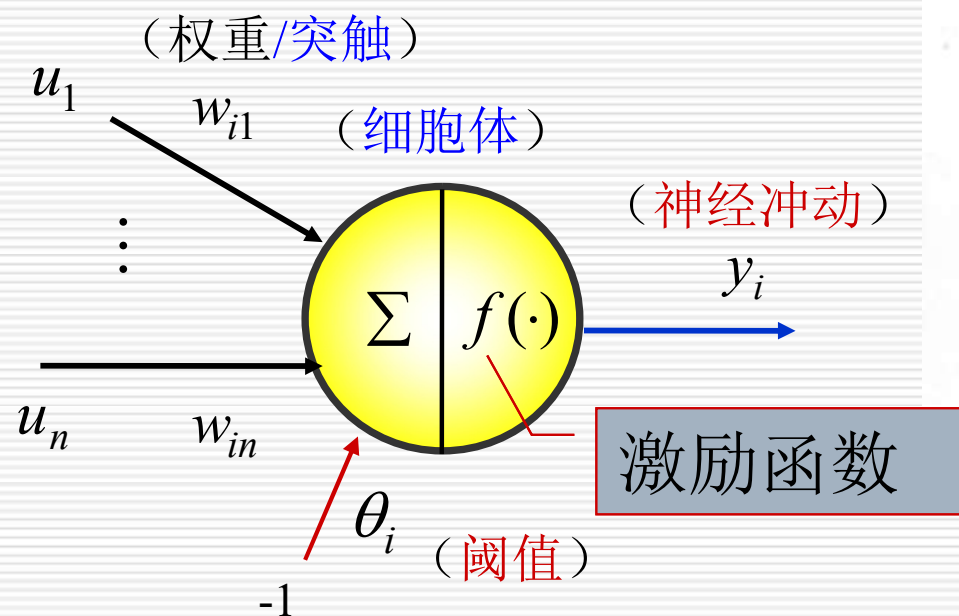
8.1 神经元与神经网络

- 8.1.1 生物神经元的结构
- 8.1.2 神经元数学模型
- 8.1.3 神经网络的结构与工作方式
- 8.1.4 神经网络的学习

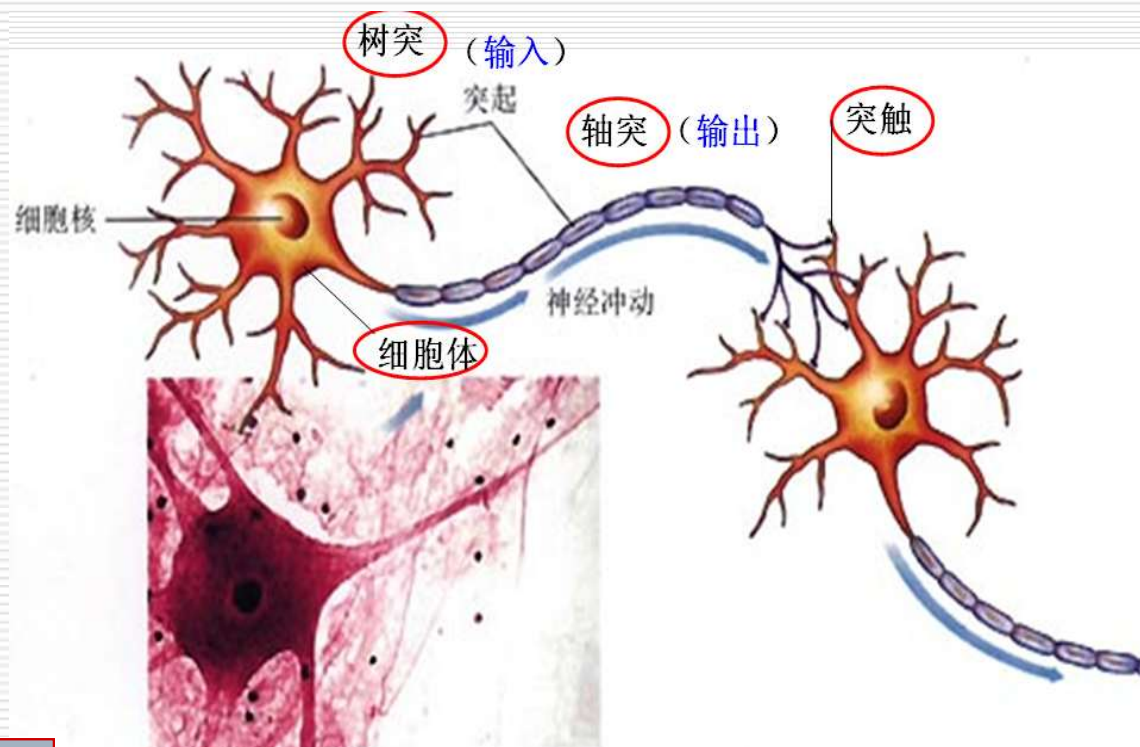
8.1.2 神经元数学模型

□ 2. 人工神经元模型

1943年，麦克洛奇和皮兹提出M—P模型。



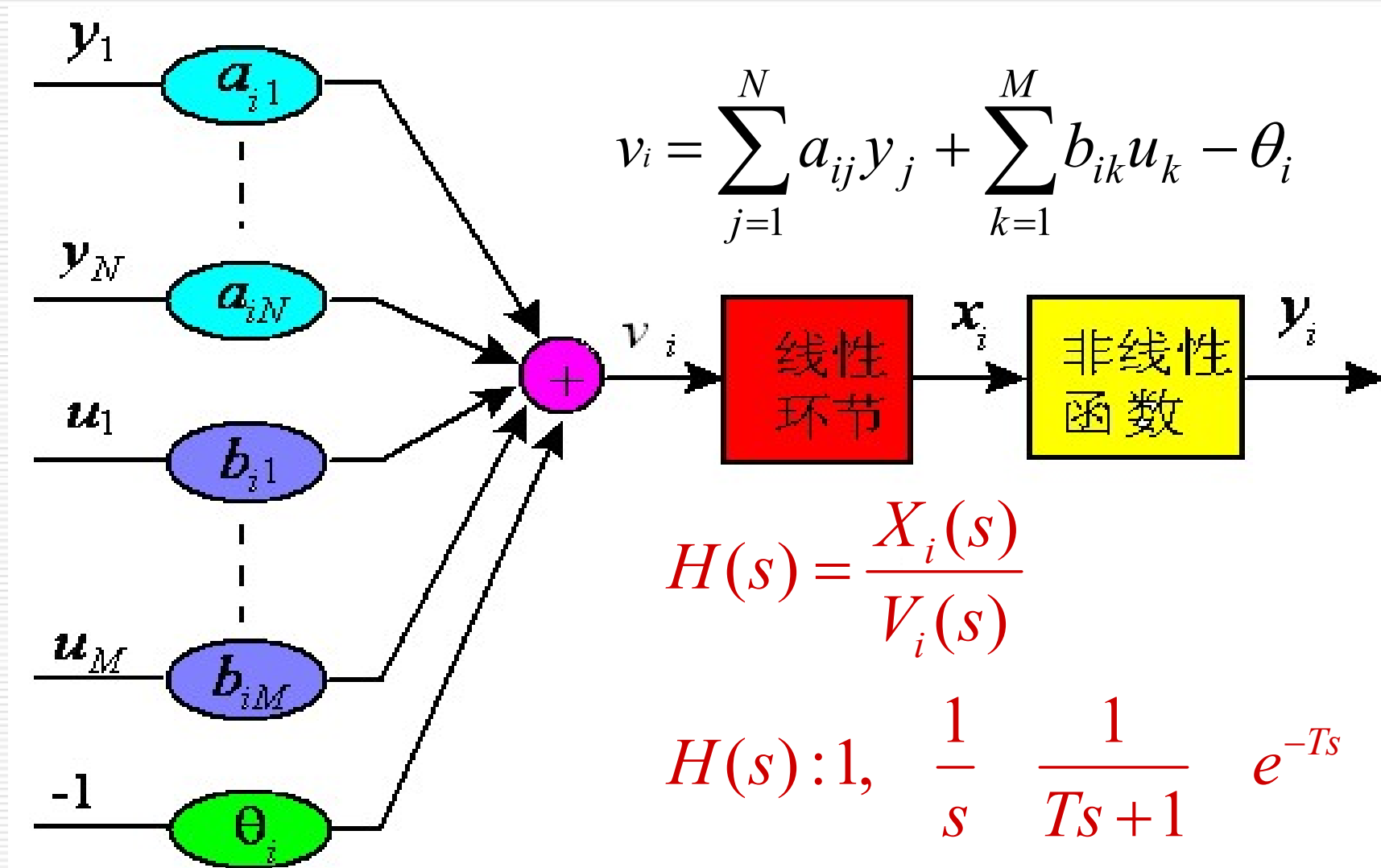
人工神经元 i 的模型图



$$y_i = f\left(\sum_{j=1}^n w_{ij} u_j - \theta_i\right) = f\left(\sum_{j=0}^n w_{ij} u_j\right)$$
$$(u_0 = -1, w_{i0} = \theta_i)$$

8.1.2 神经元数学模型

一般模型：



8.1.2 神经元数学模型

$y_i(t)$: 第 i 个神经元的输出。

θ_i : 第 i 个神经元的阈值。

$u_k(t) (k = 1, 2, \dots, M)$: 外部输入。

a_{ij}, b_{ik} : 权值。

■ 加权求和:

$$v_i(t) = \sum_{j=1}^N a_{ij} y_j(t) + \sum_{k=1}^M b_{ik} u_k(t) - \theta_i$$

其矩阵形式: $V(t) = AY(t) + BU(t) - \theta$

$$A = \{a_{ij}\}_{N \times N}$$

$$B = \{b_{ik}\}_{N \times M}$$

$$V = [v_1 \quad \cdots \quad v_N]^T$$

$$U = [u_1 \quad \cdots \quad u_M]^T$$

$$\theta = [\theta_1 \quad \cdots \quad \theta_N]^T$$

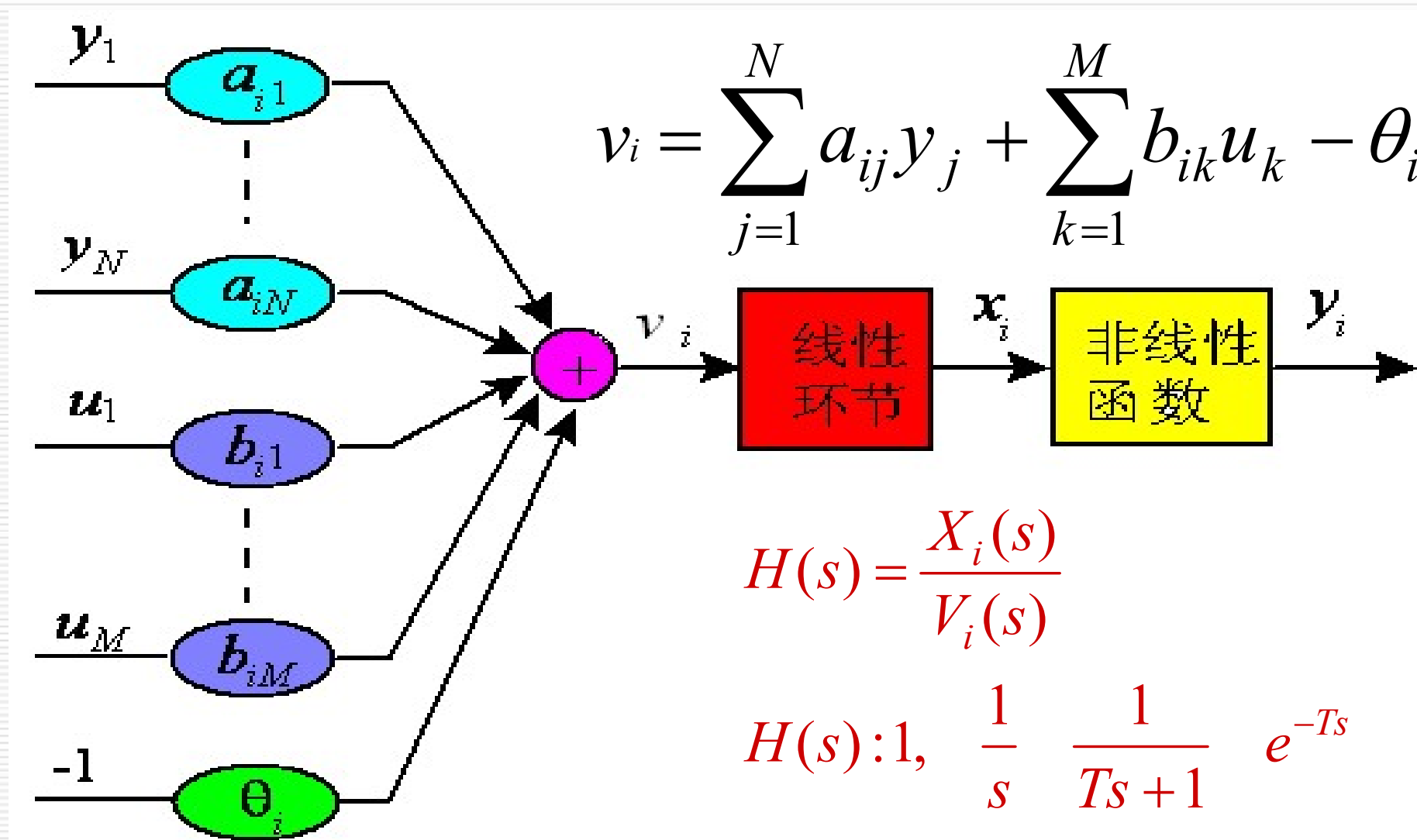
$$Y = [y_1 \quad \cdots \quad y_N]^T$$

8.1.2 神经元数学模型

- 线性环节的传递函数: $X_i(s) = H(s)V_i(s)$

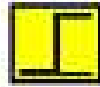
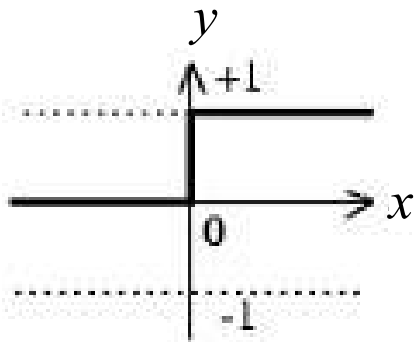
$H(s)$: 1 ; $\frac{1}{s}$; $\frac{1}{Ts+1}$; e^{-Ts} 及其组合等。

8.1.2 神经元数学模型



8.1.2 神经元数学模型

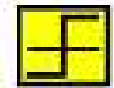
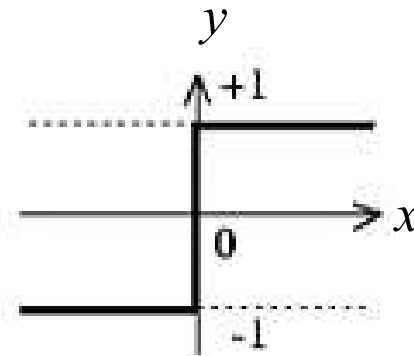
■ 非线性激励函数（传输函数、输出变换函数）



$$y = \text{hardlim}(x) = \begin{cases} 1 & x \geq 0 \\ 0 & x < 0 \end{cases}$$

Hard-Limit Transfer Function

（硬极限函数或阶跃函数）



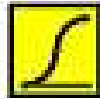
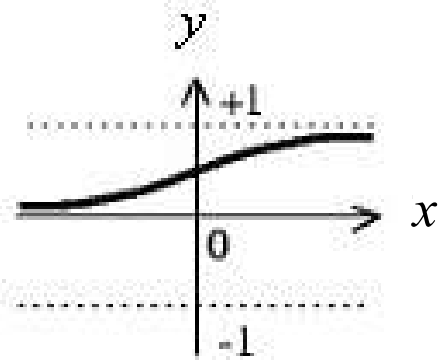
$$y = \text{hardlims}(x) = \begin{cases} 1 & x \geq 0 \\ -1 & x < 0 \end{cases}$$

Symmetric Hard-Limit Trans. Funct.

（对称硬极限函数）

8.1.2 神经元数学模型

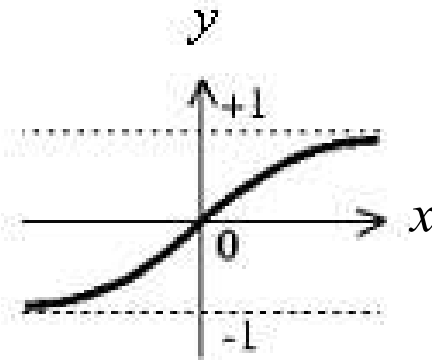
- 非线性激励函数（传输函数、输出变换函数）



$$y = \text{logsig}(x) = \frac{1}{1 + e^{-\alpha x}}$$
$$\alpha = 1$$

Log-Sigmoid Transfer Function

（对数-S 形函数或S型函数）



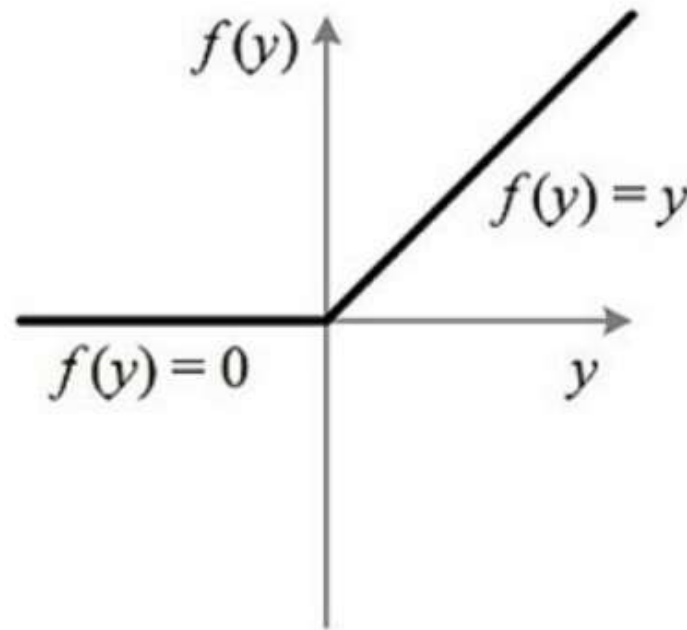
$$y = \text{tansig}(x) = \frac{e^{\alpha x} - e^{-\alpha x}}{e^{\alpha x} + e^{-\alpha x}}$$
$$\alpha = 1$$

Tan-Sigmoid Transfer Function

（双曲正切S形函数）

8.1.2 神经元数学模型

- 非线性激励函数（传输函数、输出变换函数）



$$ReLU(x) = \begin{cases} x & \text{if } x > 0 \\ 0 & \text{if } x \leq 0 \end{cases}$$

（**RELU**函数）

8.1.2 神经元数学模型

■ 工作过程:

- 从各输入端接收输入信号 u_j ($j = 1, 2, \dots, n$)
- 根据连接权值求出所有输入的加权和

$$x_i = \sum_{j=1}^n w_{ij} u_j - \theta_i = \sum_{j=1}^n w_{ij} u_j + b_i = \sum_{j=0}^n w_{ij} u_j$$

$$(w_{i0} = -1, u_0 = \theta_i \text{ 或 } w_{i0} = 1, u_0 = b_i)$$

- 用非线性激励函数进行转换，得到输出

$$y_i = f(x_i) = f\left(\sum_{j=0}^n w_{ij} u_j\right)$$

8.1 神经元与神经网络

- 8.1.1 生物神经元的结构
- 8.1.2 神经元的数学模型
- 8.1.3 神经网络的结构与工作方式
- 8.1.4 神经网络的学习

8.1.3 神经网络的结构与工作方式

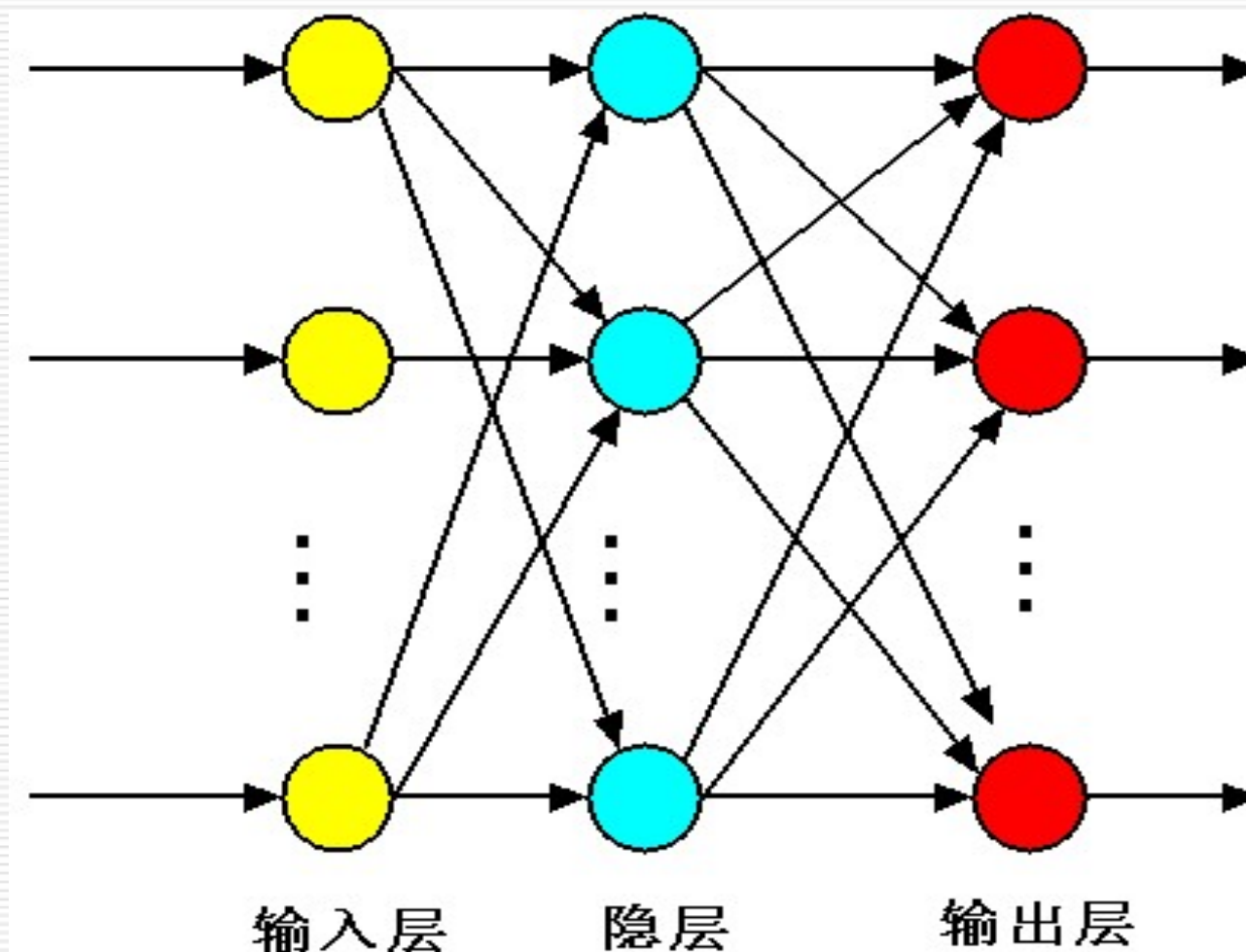
□ 决定人工神经网络性能的三大要素：

- 神经元的特性。
- 神经元之间相互连接的形式——拓扑结构。
- 为适应环境而改善性能的学习规则。

8.1.3 神经网络的结构与工作方式

□ 1. 神经网络的结构

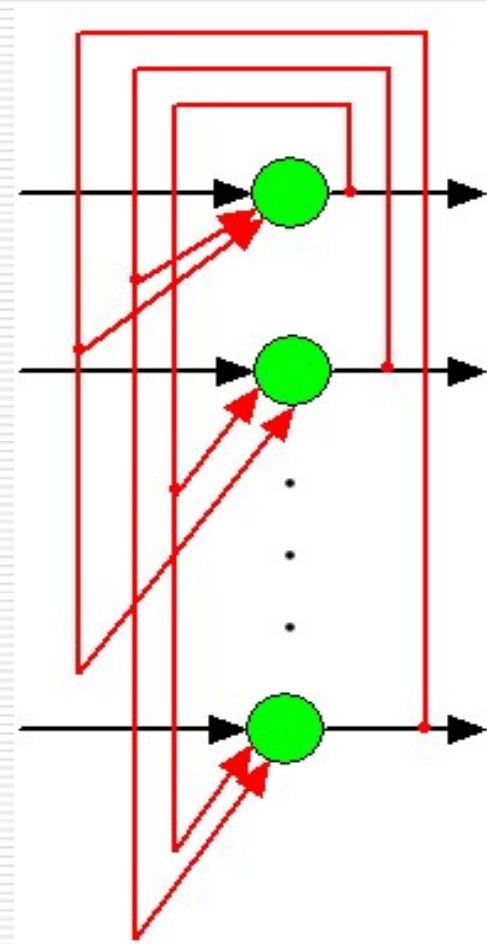
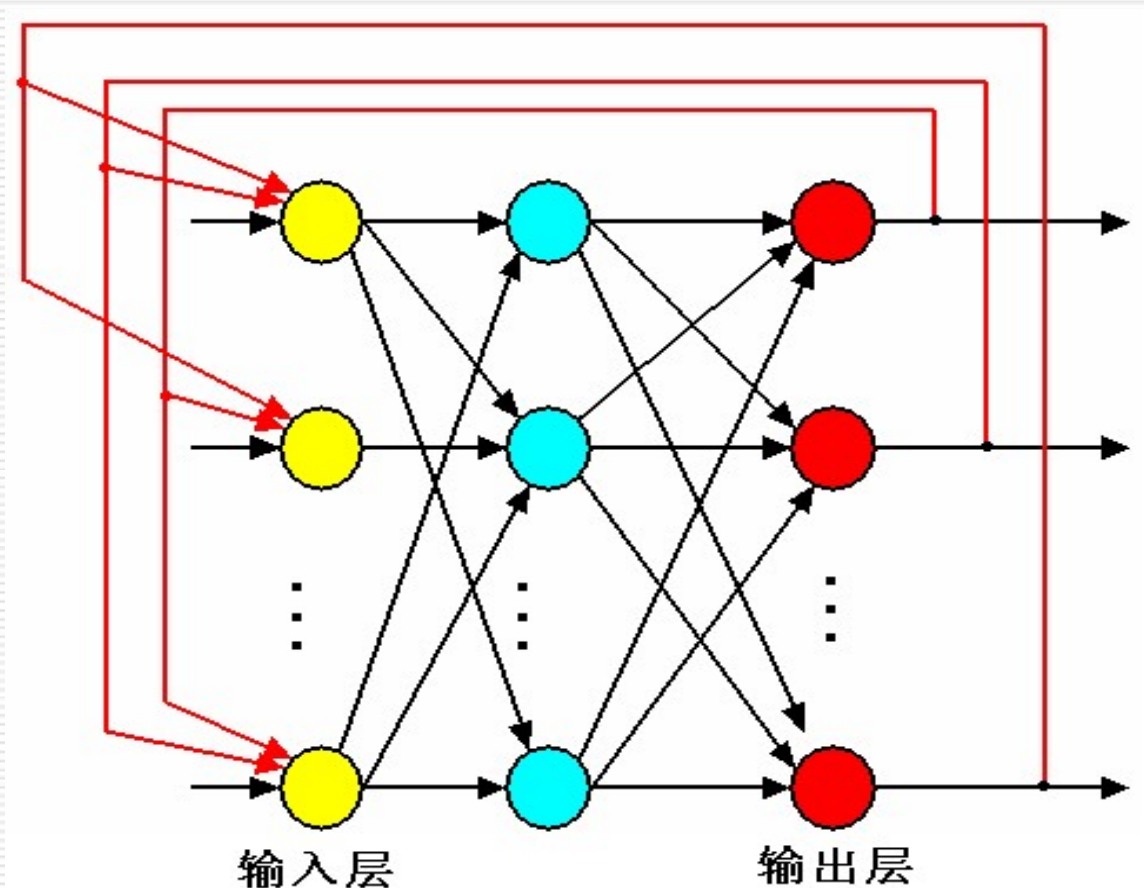
(1) 前馈型（前向型）



8.1.3 神经网络的结构与工作方式

□ 1. 神经网络的结构

(2) 反馈型



(Hopfield神经网络)

8.1.3 神经网络的结构与工作方式

□ 2. 神经网络的工作方式

- **同步**（并行）方式：任一时刻神经网络中所有神经元同时调整状态。
- **异步**（串行）方式：任一时刻只有一个神经元调整状态，而其它神经元的状态保持不变。

8.1 神经元与神经网络

- 8.1.1 生物神经元的结构
- 8.1.2 神经元的数学模型
- 8.1.3 神经网络的结构与工作方式
- 8.1.4 神经网络的学习

8.1 神经元与神经网络

□ 8.1.4 神经网络的学习

神经网络方法是一种知识表示方法和推理方法。
神经网络知识表示是一种隐式的表示方法。

1944年赫布（Hebb）提出了改变神经元连接强度的**Hebb学习规则**。

Hebb学习规则：当某一突触两端的神经元同时处于兴奋状态，那么该连接的权值应该增强。

第8章 人工神经网络及其应用

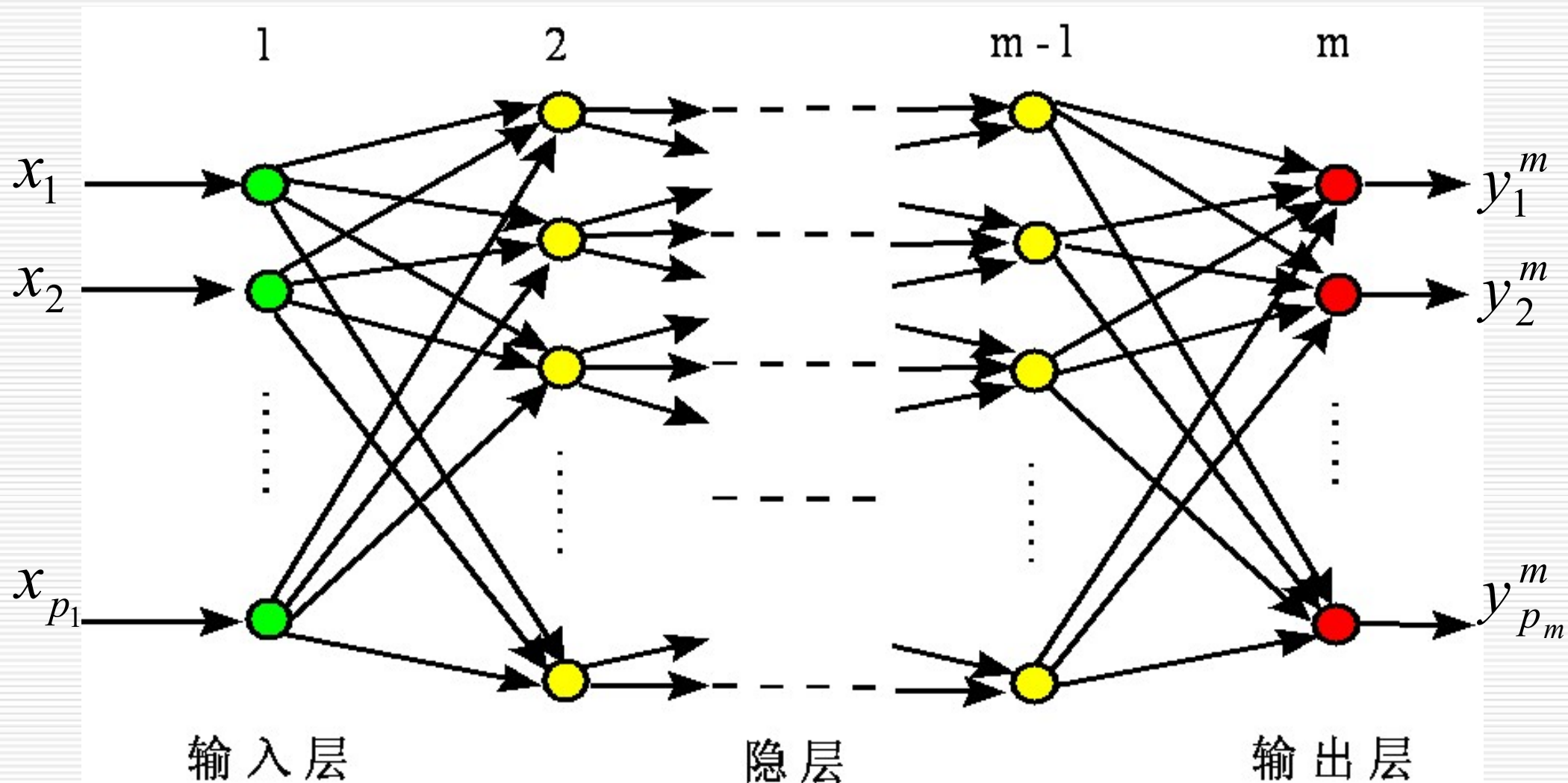
- 8.1 神经元与神经网络
- ✓ 8.2 **BP神经网络及其学习算法**
- 8.3 **BP神经网络在模式识别中的应用**
- 8.4 **Hopfield神经网络及其改进**
- 8.5 **Hopfield神经网络的应用**
- 8.6 卷积神经网络与深度学习
- 8.7 生成对抗网络及其应用

8.2 BP神经网络及其学习算法

- **8.2.1 BP神经网络的结构**
- **8.2.2 BP学习算法**
- **8.2.3 BP算法的实现**

8.2.1 BP神经网络的结构

1. BP 网络结构

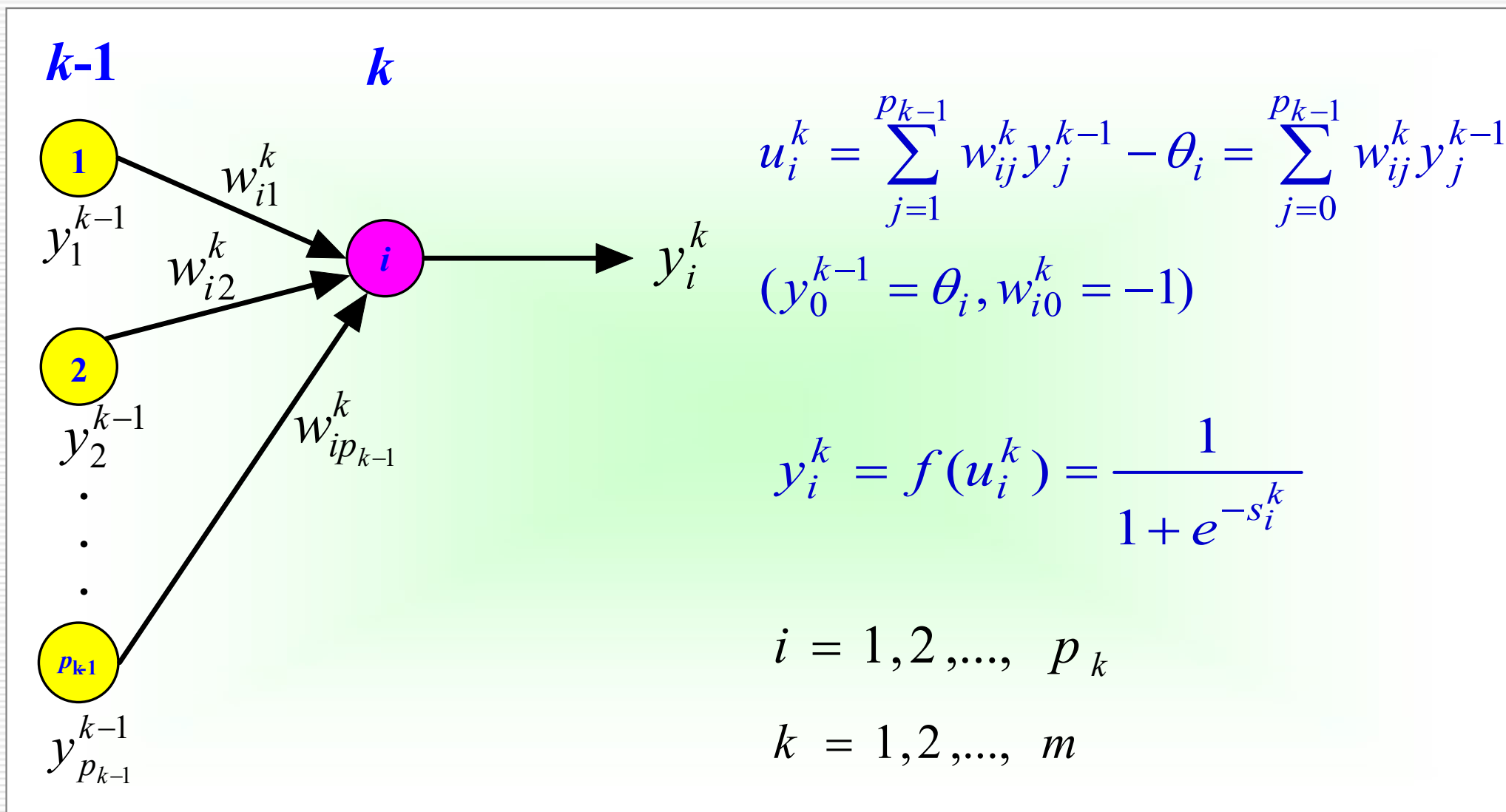


$$X = [x_1 \quad x_2 \quad \cdots \quad x_{p_1}]^T$$

$$Y = [y_1^m \quad y_2^m \quad \cdots \quad y_{p_m}^m]^T$$

8.2.1 BP神经网络的结构

2. 输入输出变换关系



8.2.1 BP神经网络的结构

3. 工作过程

- 第一阶段或网络训练阶段:

■ N 组输入输出样本: $x_i = [x_{i1}, x_{i2}, \dots, x_{ip1}]^T$

$$d_i = [d_{i1}, d_{i2}, \dots, d_{ipm}]^T$$

$$i = 1, 2, \dots, N$$

■ 对网络的连接权进行学习和调整, 以使该网络实现给定样本的输入输出映射关系。

- 第二阶段或称工作阶段: 把实验数据或实际数据输入到网络, 网络在误差范围内预测计算出结果。

8.2 BP神经网络及其学习算法

- 8.2.1 BP神经网络的结构
- 8.2.2 BP学习算法
- 8.2.3 BP算法的实现
- 8.2.3 BP算法的应用

8.2.2 BP学习算法

- 两个问题：

- (1) 是否存在一个BP神经网络能够逼近给定的样本或者函数。

BP定理： 给定任意 $\varepsilon > 0$ ， 对于任意的连续函数， 存在一个三层前向神经网络， 它可以在任意 ε 平方误差精度内逼近连续函数。

- (2) 如何调整BP神经网络的连接权， 使网络的输入与输出与给定的样本相同。

- ◆ 1986年， 鲁梅尔哈特 (D. Rumelhart) 等提出BP学习算法。

8.2.2 BP学习算法

■ 1. 基本思想

- 目标函数:
$$J = \frac{1}{2} \sum_{j=1}^{p_m} (y_j^m - d_j)^2$$

- 约束条件:
$$u_i^k = \sum_j w_{ij}^{k-1} y_j^{k-1} \quad i = 1, 2, \dots, p_k$$

$$y_i^k = f_k(u_i^k) \quad k = 1, 2, \dots, m$$

- 连接权值的修正量:

$$\Delta w_{ij}^{k-1} = -\varepsilon \frac{\partial J}{\partial w_{ij}^{k-1}} \quad j = 1, 2, \dots, p_{k-1}$$

8.2.2 BP学习算法

先求
$$\frac{\partial J}{\partial w_{ij}^{k-1}} = \frac{\partial J}{\partial u_i^k} \frac{\partial u_i^k}{\partial w_{ij}^{k-1}} = \frac{\partial J}{\partial u_i^k} \frac{\partial}{\partial w_{ij}^{k-1}} \left(\sum_j w_{ij}^{k-1} y_j^{k-1} \right) = \frac{\partial J}{\partial u_i^k} y_j^{k-1}$$

记
$$d_i^k = \frac{\partial J}{\partial u_i^k} = \frac{\partial J}{\partial y_i^k} \frac{\partial y_i^k}{\partial u_i^k} = \frac{\partial J}{\partial y_i^k} f'_k(u_i^k)$$

(1) 对输出层的神经元
$$\frac{\partial J}{\partial y_i^k} = \frac{\partial J}{\partial y_i^m} = y_i^m - y_{si}$$

$$d_i^m = (y_i^m - y_{si}) f'_m(u_i^m)$$

(2) 对隐单元层, 则有
$$\frac{\partial J}{\partial y_i^k} = \sum_l \frac{\partial J}{\partial u_l^{k+1}} \frac{\partial u_l^{k+1}}{\partial y_i^k} = \sum_l d_l^{k+1} w_{li}^k$$

$$d_i^k = f'_k(u_i^k) \sum_l d_l^{k+1} w_{li}^k$$

$$\Delta w_{ij}^{k-1} = -\varepsilon d_i^k y_j^{k-1}$$

8.2.2 BP学习算法

■ 2. BP学习算法

$$\Delta w_{ij}^{k-1} = -\varepsilon d_i^k y_j^{k-1}$$

$$d_i^m = (y_i^m - y_{si}) f'_m(u_i^m)$$

——输出层连接权调整公式

$$d_i^k = f'_k(u_i^k) \sum_l d_l^{k+1} w_{li}^k$$

——隐层连接权调整公式

8.2.2 BP学习算法

■ 2. 学习算法

$$\text{当 } y_i^k = \frac{1}{1 + e^{-u_i^k}} \text{ 时}$$

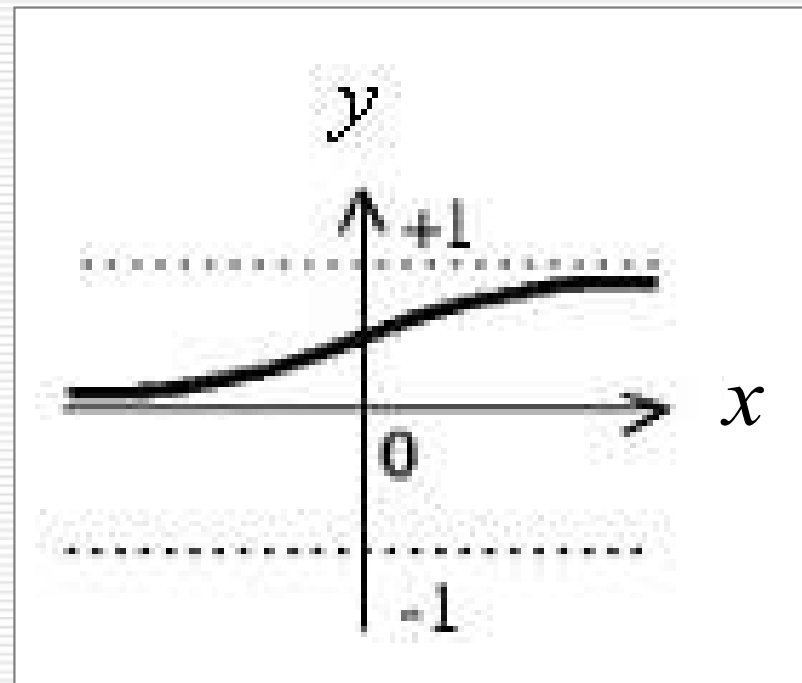
$$\Delta w_{ij}^{k-1} = -\varepsilon d_i^k y_j^{k-1}$$

$$d_i^m = y_i^m (1 - y_i^m) (y_i^m - y_i)$$

——输出层连接权调整公式

$$d_i^k = y_i^k (1 - y_i^k) \sum_{l=1}^{p_{k+1}} w_{li}^k d_l^{k+1}$$

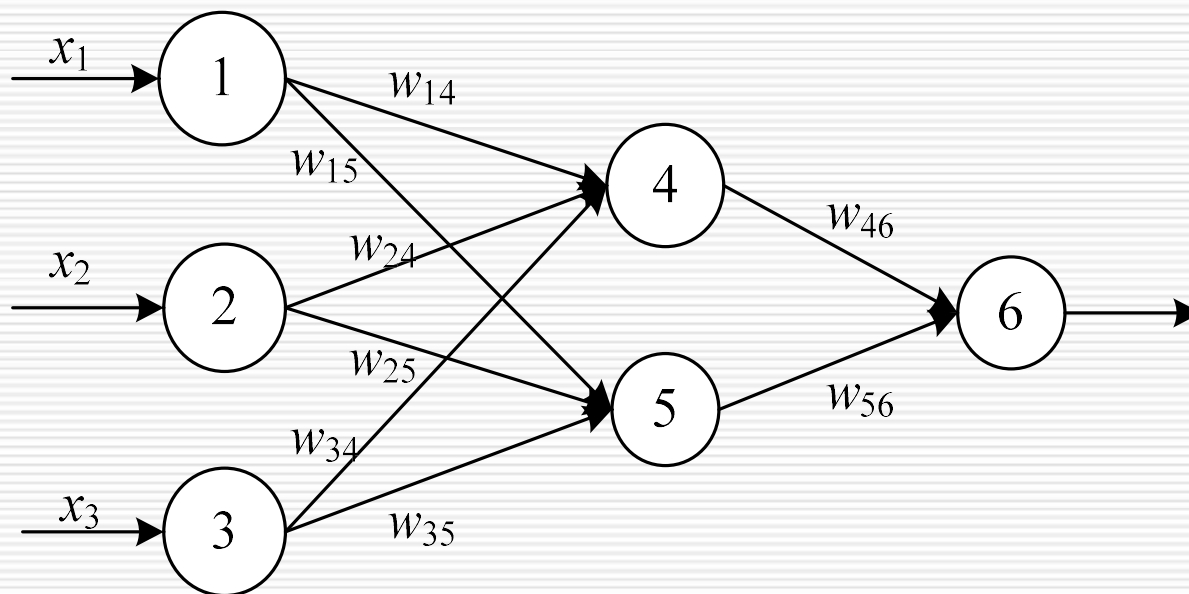
——隐层连接权调整公式



8.2.2 BP学习算法

- 例8.2 已知一个前馈型神经网络例子如下图所示。设学习率 l 为0.9，当前的训练样本为 $x=\{1,0,1\}$ ，而且预期分类标号为1，同时，下表给出了当前该网络的各个连接权值和神经元偏置。求该网络在当前训练样本下的训练过程。

w_{14}	w_{15}	w_{24}	w_{25}	w_{34}	w_{35}	w_{46}	w_{56}	θ_4	θ_5	θ_6
0.2	-0.3	0.4	0.1	-0.5	0.2	-0.3	-0.2	-0.4	0.2	0.1



8.2.2 BP学习算法

w_{14}	w_{15}	w_{24}	w_{25}	w_{34}	w_{35}	w_{46}	w_{56}	θ_4	θ_5	θ_6
0.2	-0.3	0.4	0.1	-0.5	0.2	-0.3	-0.2	-0.4	0.2	0.1

第一步：前向计算每个神经元的输入加权和 S_j 和输出 O_j

单元 j	加权和 S_j	输出 O_j
4	$w_{14} \times x_1 + w_{24} \times x_2 + w_{34} \times x_3 + \theta_4 = 0.2 \times 1 + 0.4 \times 0 + (-0.5) \times 1 + (-0.4) = -0.7$	$1/(1+e^{-(0.7)})=0.332$
5	$w_{15} \times x_1 + w_{25} \times x_2 + w_{35} \times x_3 + \theta_5 = -0.3 \times 1 + 0.1 \times 0 + 0.2 \times 1 + 0.2 = 0.1$	$1/(1+e^{-0.1})=0.525$
6	$w_{46} \times O_4 + w_{56} \times O_5 + \theta_6 = -0.3 \times 0.332 + (-0.2) \times 0.525 + 0.1 = -0.105$	$1/(1+e^{-(0.105)})=0.474$

8.2.2 BP学习算法

w_{14}	w_{15}	w_{24}	w_{25}	w_{34}	w_{35}	w_{46}	w_{56}	θ_4	θ_5	θ_6
0.2	-0.3	0.4	0.1	-0.5	0.2	-0.3	-0.2	-0.4	0.2	0.1

第二步：后向传播误差 E_j

单元 j	误差 E_j (T_j 是期望输出, 本例题为1)
6	$O_j(1-O_j)(T_j-O_j) = 0.474 \times (1-0.474) \times (1-0.474) = 0.1311$
5	$O_j(1-O_j) \sum_k w_{jk} E_k = 0.525 \times (1-0.525) \times (0.1311) \times (-0.2) = -0.0065$
4	$O_j(1-O_j) \sum_k w_{jk} E_k = 0.332 \times (1-0.332) \times (0.1311) \times (-0.3) = -0.0087$

8.2.2 BP学习算法

w_{14}	w_{15}	w_{24}	w_{25}	w_{34}	w_{35}	w_{46}	w_{56}	θ_4	θ_5	θ_6
0.2	-0.3	0.4	0.1	-0.5	0.2	-0.3	-0.2	-0.4	0.2	0.1

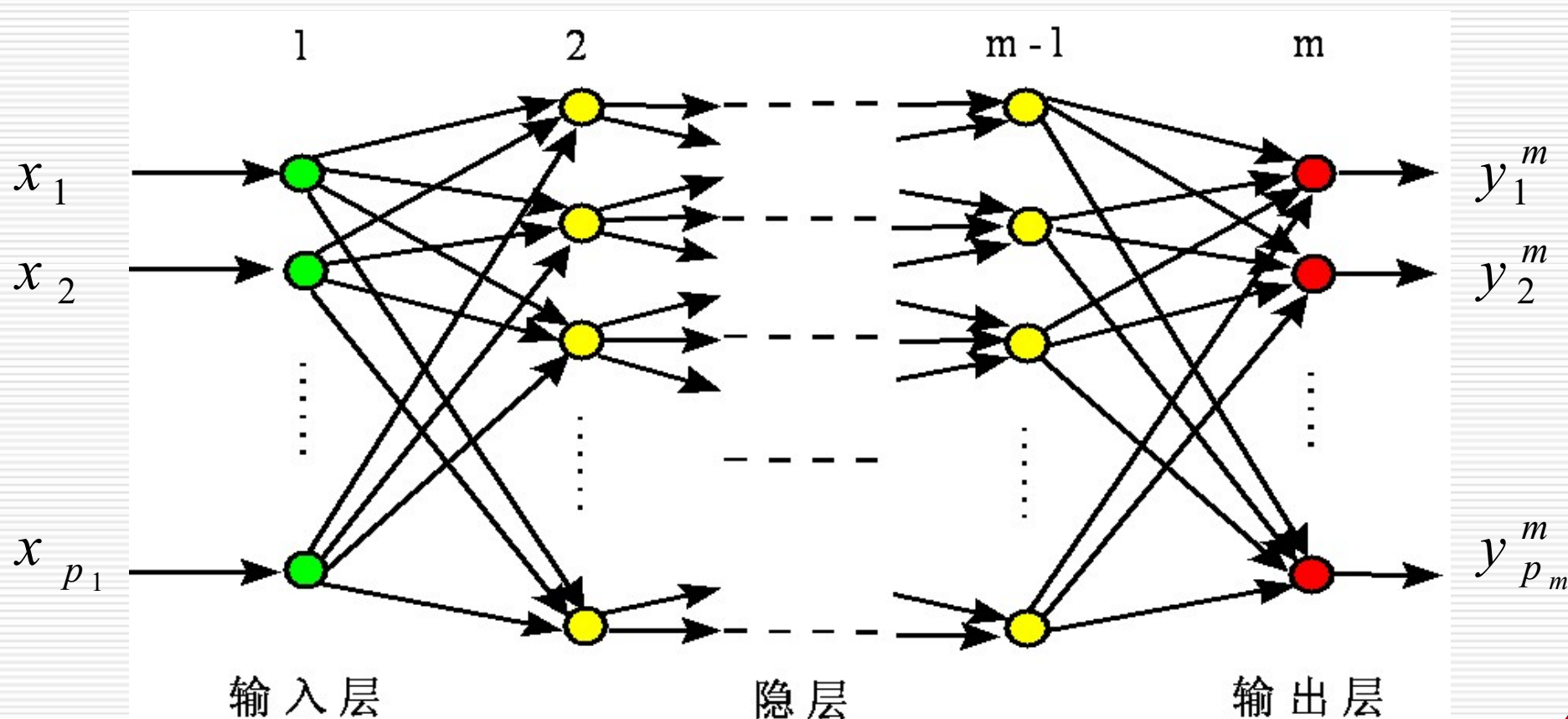
第三步：更新权值或者偏置

权值或偏置	新值
w_{46}	$\omega_{ij} + \Delta\omega_{ij} = \omega_{ij} + (l)O_iE_j = -0.3 + 0.9 \times 0.332 \times 0.1311 = -0.261$
w_{56}	$\omega_{ij} + \Delta\omega_{ij} = \omega_{ij} + (l)O_iE_j = -0.2 + 0.9 \times 0.525 \times 0.1311 = -0.138$
w_{14}	$\omega_{ij} + \Delta\omega_{ij} = \omega_{ij} + (l)O_iE_j = 0.2 + 0.9 \times 1 \times (-0.0087) = 0.192$
w_{15}	$\omega_{ij} + \Delta\omega_{ij} = \omega_{ij} + (l)O_iE_j = -0.3 + 0.9 \times 1 \times (-0.0065) = -0.306$
w_{24}	$\omega_{ij} + \Delta\omega_{ij} = \omega_{ij} + (l)O_iE_j = 0.4 + 0.9 \times 0 \times (-0.0087) = 0.4$
w_{25}	$\omega_{ij} + \Delta\omega_{ij} = \omega_{ij} + (l)O_iE_j = 0.1 + 0.9 \times 0 \times (-0.0065) = 0.1$
w_{34}	$\omega_{ij} + \Delta\omega_{ij} = \omega_{ij} + (l)O_iE_j = -0.5 + 0.9 \times 1 \times (-0.0087) = -0.508$
w_{35}	$\omega_{ij} + \Delta\omega_{ij} = \omega_{ij} + (l)O_iE_j = 0.2 + 0.9 \times 1 \times (-0.0065) = 0.194$
θ_6	$\theta_j + \Delta\theta_j = \theta_j + (l)E_j = 0.1 + 0.9 \times 0.1311 = 0.218$
θ_5	$\theta_j + \Delta\theta_j = \theta_j + (l)E_j = 0.2 + 0.9 \times (-0.0065) = 0.194$
θ_4	$\theta_j + \Delta\theta_j = \theta_j + (l)E_j = -0.4 + 0.9 \times (-0.0087) = -0.408$

8.2.2 BP学习算法

■ 2. 学习算法

- 正向传播：输入信息由输入层传至隐层，最终在输出层输出。
- 反向传播：修改各层神经元的权值，使误差信号最小。



8.2 BP神经网络及其学习算法

□ 8.2.1 BP神经网络的结构

□ 8.2.2 BP学习算法

□ 8.2.3 BP算法的实现

8.2.3 BP算法的实现

■ 1. BP算法的设计

- (1) **隐层数及隐层神经元数的确定**：目前尚无理论指导。
- (2) **初始权值的设置**：一般以一个均值为0的随机分布设置网络的初始权值。
- (3) **训练数据预处理**：线性的特征比例变换，将所有特征变换到[0, 1]或者[-1, 1]区间内，使得在每个训练集上，每个特征的均值为0，并且具有相同的方差。
- (4) **后处理过程**：当应用神经网络进行分类操作时，通常将输出值编码成所谓的名义变量，具体的值对应类别标号。

8.2.3 BP算法的实现

■ 2. BP算法的计算机实现流程

(1) 初始化：对所有连接权和阈值赋以随机任意小值；

$$w_{ij}^k(t), \quad \theta_i^k(t), (k = 1, \dots, m; i = 1, \dots, p_k; j = 1, \dots, p_{k-1}; t = 0)$$

(2) 从 N 组输入输出样本中取一组样本： $\mathbf{x}=[x_1, x_2, \dots, x_{p_1}]^T$, $\mathbf{d}=[d_1, d_2, \dots, d_{p_m}]^T$, 把输入信息 $\mathbf{x}=[x_1, x_2, \dots, x_{p_1}]^T$ 输入到BP网络中

(3) 正向传播：计算各层节点的输出：

$$y_i^k \quad (i=1, \dots, p_k; k=1, \dots, m)$$

(4) 计算网络的实际输出与期望输出的误差：

$$e_i = y_i - y_i^m \quad (i = 1, \dots, p_m)$$

8.2.3 BP算法的实现

■ 2. BP算法的计算机实现流程

(5) 反向传播：从输出层方向计算到第一个隐层，按连接权值修正公式向减小误差方向调整网络的各个连接权值。

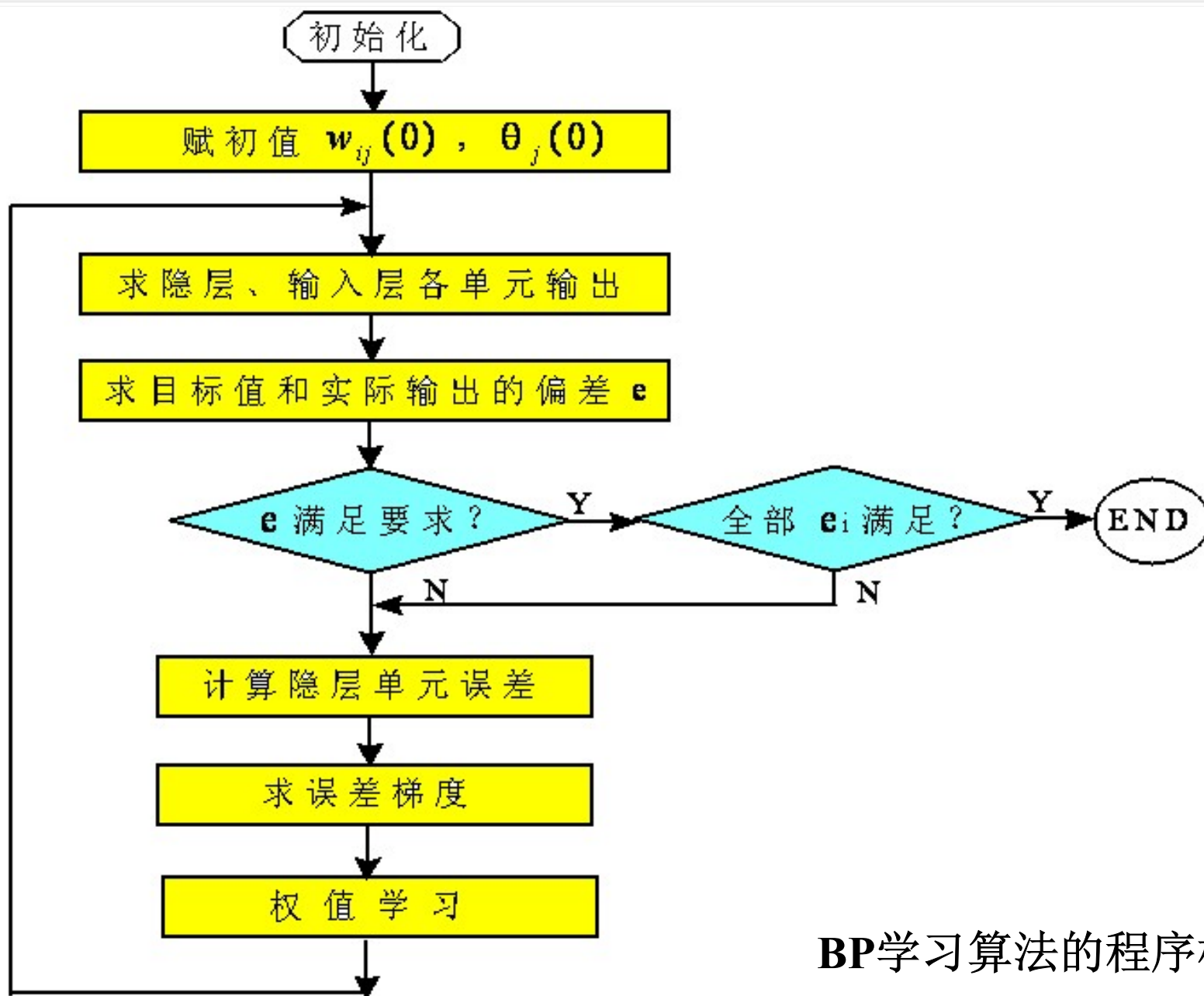
$$\Delta w_{ij} = -\alpha d_i^k y_j^{k-1}$$

$$d_i^m = y_i^m (1 - y_i^m) (y_i^m - y_i) \quad \text{——输出层连接权调整公式}$$

$$d_i^k = y_i^k (1 - y_i^k) \sum_{l=1}^{p_{k+1}} w_{li}^k d_l^{k+1} \quad \text{——隐层连接权调整公式}$$

(6) 让 $t+1 \rightarrow t$ ，取出另一组样本重复 (2) — (5)，直到 N 组输入输出样本的误差达到要求时为止。

8.2.3 BP算法的实现



BP学习算法的程序框图

BP算法的特点分析

□ 1. 特点

- **BP网络**：多层前向网络（输入层、隐层、输出层）。
- **连接权值**：通过**Delta**学习算法进行修正。
- **神经元传输函数**：S形函数。
- **学习算法**：正向传播、反向传播。
- **层与层的连接**是单向的，信息的传播是双向的。

BP算法的特点分析

□ 2. BP网络的主要优缺点

■ 优点

- 很好的逼近特性。
- 具有较强的泛化能力。
- 具有较好的容错性。

■ 缺点

- 收敛速度慢。
- 局部极值。
- 难以确定隐层和隐层结点的数目。

第8章 人工神经网络及其应用

- 8.1 神经元与神经网络
- 8.2 BP神经网络及其学习算法
- 8.3 BP神经网络在模式识别中的应用
- 8.4 Hopfield神经网络及其改进
- 8.5 Hopfield神经网络的应用
- 8.6 卷积神经网络与深度学习
- 8.7 生成对抗网络及其应用

8.3 BP神经网络在模式识别中的应用

模式识别研究用计算机模拟生物、人的感知，对模式信息，如图像、文字、语音等，进行识别和分类。

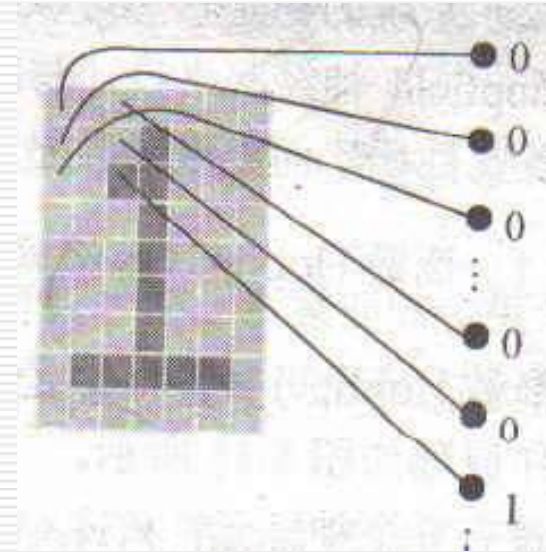
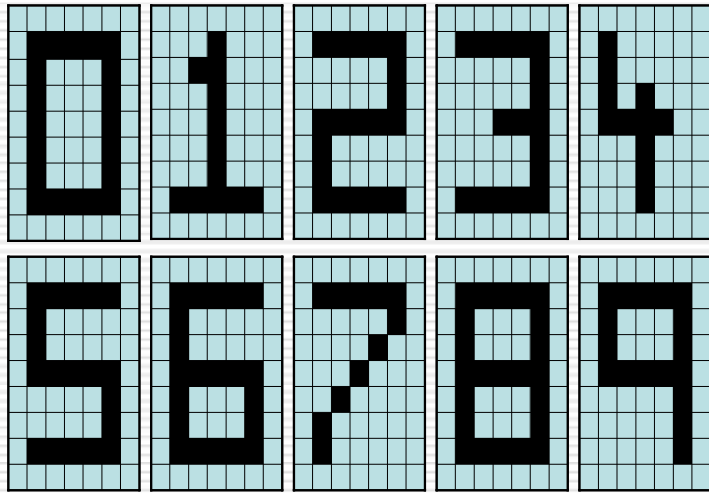
传统人工智能的研究部分地显示了人脑的归纳、推理等智能。但是，对于人类底层的智能，如视觉、听觉、触觉等方面，现代计算机系统的信息处理能力还不如一个幼儿园的孩子。

神经网络模型模拟了人脑神经网络的特点：处理单元的广泛连接；并行分布式信息储存、处理；自适应学习能力等。

神经网络模式识别方法具有较强的容错能力、自适应学习能力、并行信息处理能力。

8.3 BP神经网络在模式识别中的应用

例8.2 设计一个三层BP网络对数字0至9进行分类。



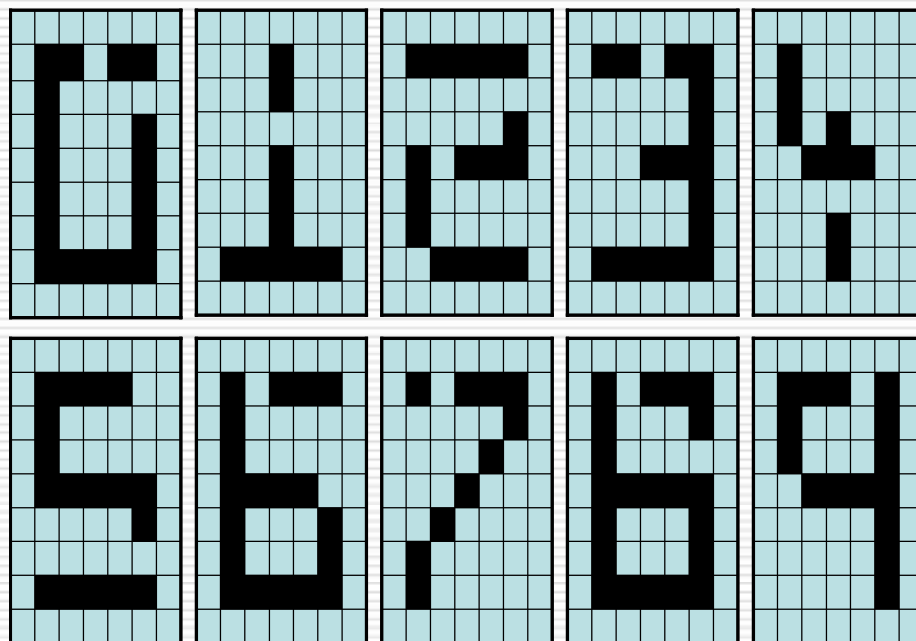
每个数字用 9×7 的网格表示，灰色像素代表0，黑色像素代表1。将每个网格表示为0, 1的长位串。位映射由左上角开始向下直到网格的整个一列，然后重复其他列。

选择BP网络结构为63-6-9。 9×7 个输入结点，对应上述网格的映射。9个输出结点对应10种分类。

使用的学习步长为0.3。训练1000个周期，如果输出结点的值大于0.9，则取为1，如果输出结点的值小于0.1，则取为0。

8.3 BP神经网络在模式识别中的应用

当训练成功后，对如图所示测试数据进行测试。测试数据都有一个或者多个位丢失。



测试结果表明：除了8以外，所有被测的数字都能够被正确地识别。

对于数字8，神经网络的第6个结点的输出值为1，第8个结点的输出值为0.49，表明第8个样本是模糊的，可能是数字6，也可能是数字8，但也不完全确信是两者之一。

第8章 人工神经网络及其应用

- 8.1 神经元与神经网络
- 8.2 BP神经网络及其学习算法
- 8.3 BP神经网络在模式识别中的应用
- 8.4 Hopfield神经网络及其改进
- 8.5 Hopfield神经网络的应用
- 8.6 卷积神经网络与深度学习
- 8.7 生成对抗网络及其应用

8.4 Hopfield神经网络及其改进

- 8.4.1 离散型Hopfield神经网络
- 8.4.2 连续型Hopfield神经网络及其VLSI实现
- 8.4.3 随机神经网络

8.4 Hopfield神经网络及其改进

- 8.4.1 离散型Hopfield神经网络
- 8.4.2 连续型Hopfield神经网络及其VLSI实现
- 8.4.3 随机神经网络

8.4 Hopfield神经网络及其改进

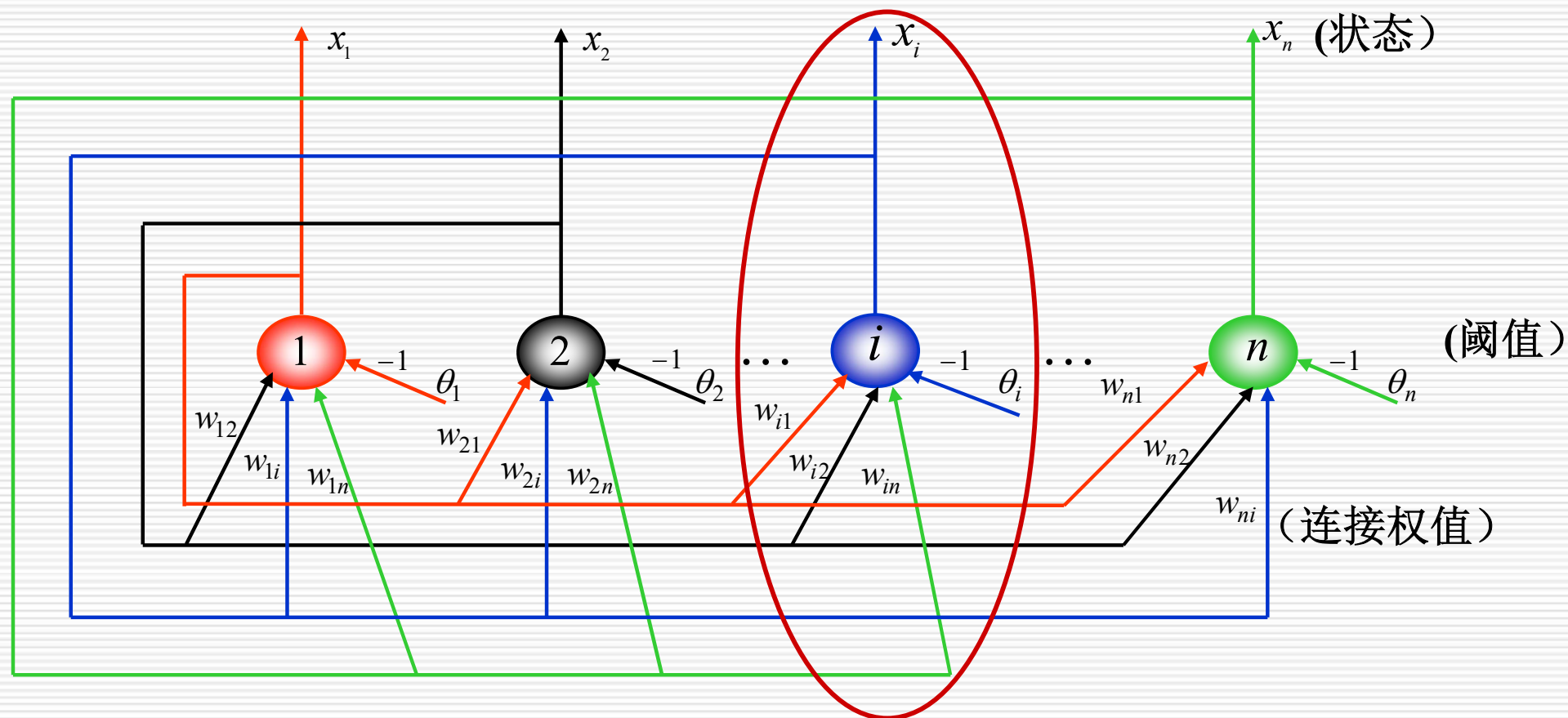


J. J. Hopfield

8.4.1 离散Hopfield神经网络

1. 离散Hopfield神经网络模型

■ 网络结构:

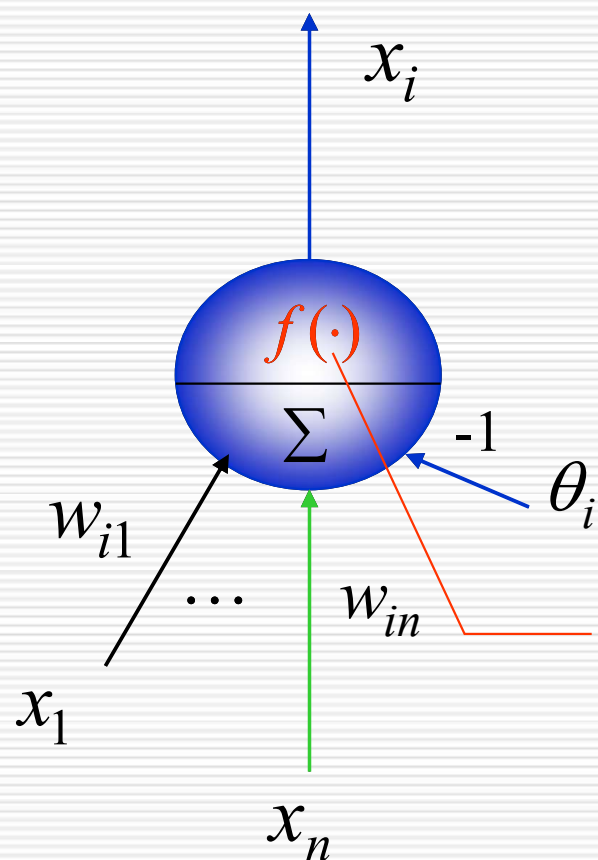


离散Hopfield神经网络结构图

8.4.1 离散Hopfield 神经网络

1. 离散Hopfield神经

■ 输入输出关系:



$$x(k+1) = f(W_x(k) - \theta), \forall i$$

$$\mathbf{x} = [x_1 \ x_2 \ \cdots \ x_n]^T \quad \boldsymbol{\theta} = [\theta_1 \ \theta_2 \ \cdots \ \theta_n]^T$$

$$\mathbf{W} = [w_{ij}]_{n \times n} \quad \mathbf{f}(s) = [f(s_1) \ f(s_2) \ \cdots \ f(s_n)]^T$$

$$x_i(k+1) = f\left(\sum_{j=1}^n w_{ij} x_j(k) - \theta_i\right)$$

注: $w_{ii}=0$

$$f(s) = \begin{cases} 1 & s \geq 0 \\ -1 & s < 0 \end{cases} \quad \text{或} \quad f(s) = \begin{cases} 1 & s \geq 0 \\ 0 & s < 0 \end{cases}$$

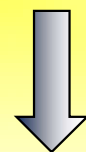
8.4.1 离散Hopfield神经网络

1. 离散Hopfield神经网络模型

■ 工作方式:

异步（串行）方式:
$$\begin{cases} x_i(k+1) = f\left(\sum_{j=1}^n w_{ij}x_j(k) - \theta_i\right) \\ x_j(k+1) = x_j(k), \quad j \neq i \end{cases}$$

同步（并行）方式:
$$x_i(k+1) = f\left(\sum_{j=1}^n w_{ij}x_j(k) - \theta_i\right), \forall i$$

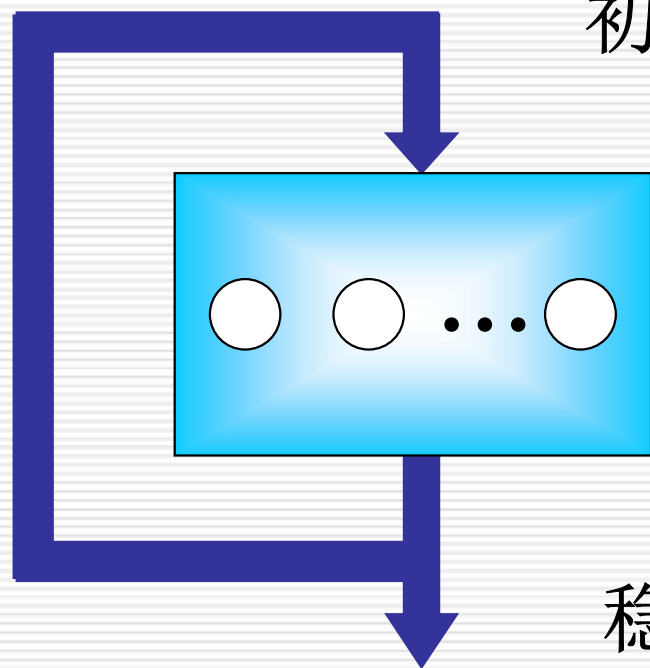


$$x(k+1) = f(W_x(k) - \theta), \forall i$$

8.4.1 离散Hopfield神经网络

1. 离散Hopfield神经网络模型

- 工作过程:



初态: $x(0) = [x_1(0), x_2(0), \dots, x_n(0)]^T$

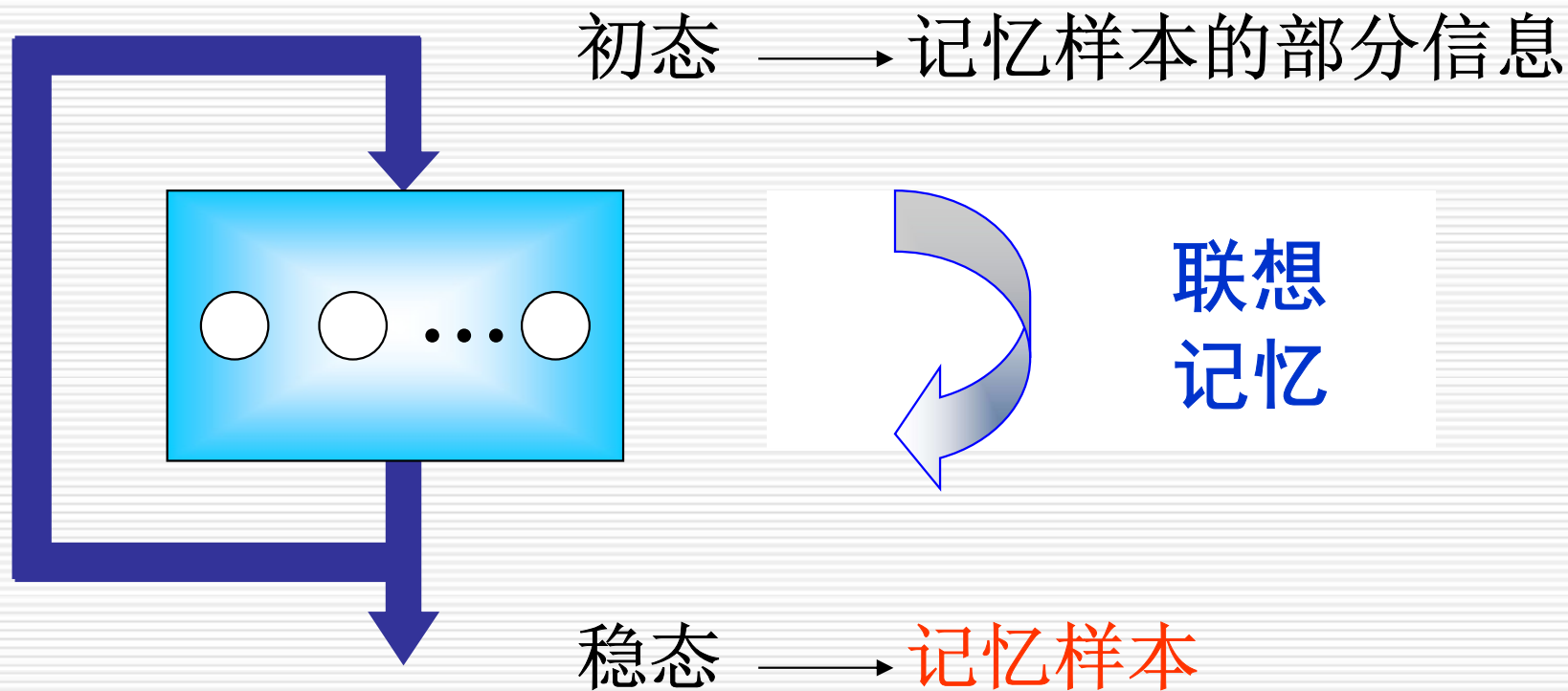
(异步或同步方式)

稳态: $\lim_{k \rightarrow \infty} x(k)$

8.4.1 离散Hopfield神经网络

1. 离散Hopfield神经网络模型

- 工作过程:



8.4.1 离散Hopfield 神经网络

2. 网络的稳定性

- 稳定性定义:

- 若从某一时刻开始, 网络中所有神经元的状态不再改变, 即 $x = f(w_x - \theta)$, 则称该网络是稳定的, x 为网络的稳定点或吸引子。

- Hopfield神经网络是高维非线性系统, 可能有许多稳定状态。从任何初始状态, 网络最终都会收敛到一个稳定状态。这些稳定状态可以

$$x(k) = f(W_x(k) - \theta) = x(k + 1)$$

8.4.1 离散Hopfield 神经网络

2. 网络的稳定性

- 稳定性定理证明：1983年，科恩（Cohen）、葛劳斯伯格（S. Grossberg）。
- 稳定性定理（Hopfield）
 - 串行稳定性 —— W ：对称阵
 - 并行稳定性 —— W ：非负定对称阵

8.4 Hopfield神经网络及其改进

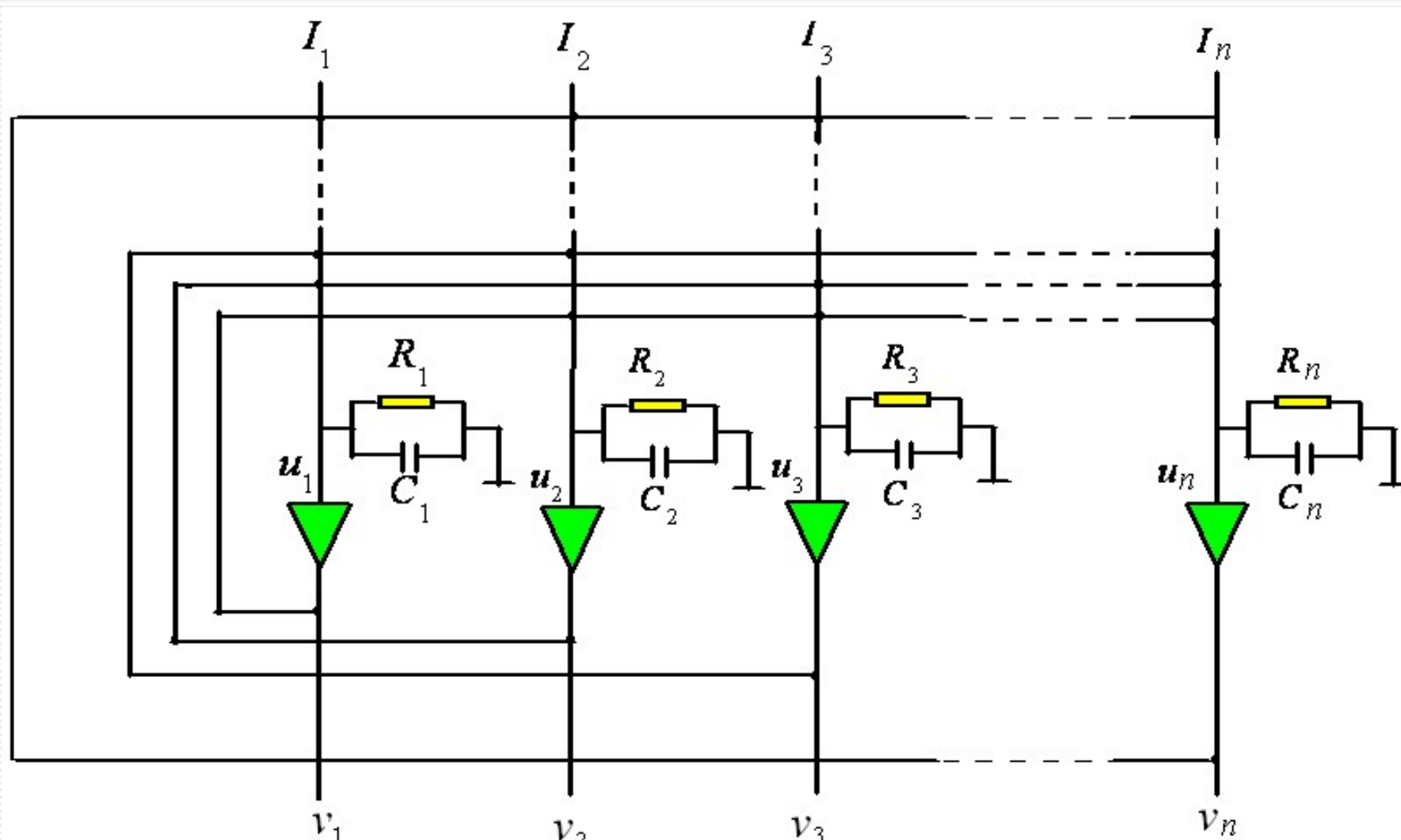
- 8.4.1 离散型Hopfield神经网络
- 8.4.2 连续型Hopfield神经网络及其VLSI实现

8.4.2 连续型Hopfield神经网络及其VLSI实现

- 连续Hopfield神经网络模型
- 网络的稳定性
- 应用举例

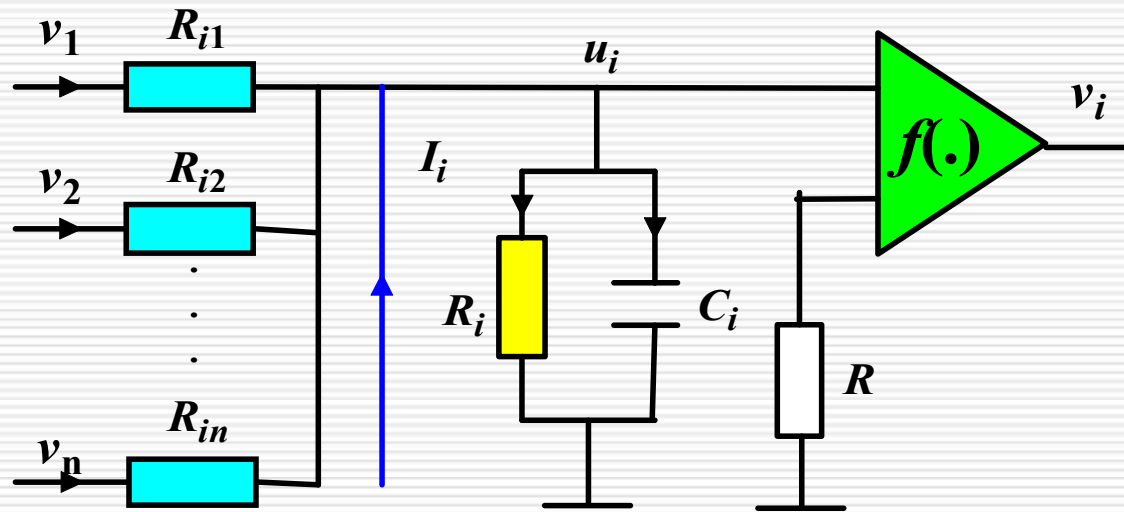
8.4.2 连续型Hopfield神经网络及其VLSI实现

1. 连续Hopfield神经网络模型



8.4.2 连续型Hopfield神经网络及其VLSI实现

1. 连续Hopfield神经网络模型



$$\begin{cases} \frac{u_i}{R_i} + C_i \frac{du_i}{dt} = \sum_{j=1}^n \frac{v_j - u_i}{R_{ij}} + I_i \\ v_i = f(u_i) \end{cases}$$

$$\frac{1}{R'_i} = \frac{1}{R_i} + \sum_{j=1}^n \frac{1}{R_{ij}}$$

$$w_{ij} = \frac{1}{R_{ij}}$$

$$C_i \frac{du_i}{dt} = -\frac{u_i}{R'_i} + \sum_{j=1}^n w_{ij} v_j + I_i$$

$$v_i = f(u_i) = \frac{1}{1 + e^{-\frac{2u_i}{u_0}}} = \frac{1}{2} \left[1 + \tanh\left(\frac{u_i}{u_0}\right) \right]$$

8.4.2 连续型Hopfield神经网络及其VLSI实现

2. 网络的稳定性

- 计算能量函数：

$$E = -\frac{1}{2} \sum_{i=1}^n \sum_{j=1}^n w_{ij} v_i v_j - \sum_{i=1}^n v_i I_i + \sum_{i=1}^n \frac{1}{R'_i} \int_0^{v_i} f^{-1}(v) dv$$

- 定理：对于连续型 **Hopfield** 神经网络，若 $f^{-1}(\cdot)$ 为单调递增的连续函数， $C_i > 0$ ， $w_{ij} = w_{ji}$ ，则 $\frac{dE}{dt} \leq 0$ ；当且仅当 $\frac{dv_i}{dt} = 0$ ， $(1 \leq i \leq n)$ 时， $\frac{dE}{dt} = 0$

8.4.3 随机神经网络

- **Hopfield**神经网络中，神经元状态为1是根据其输入是否大于阈值确定的，是确定性的。
- 随机神经网络中，神经元状态为1是随机的，服从一定的概率分布。例如，服从玻尔兹曼(**Boltzmann**)、高斯(**Gaussian**)、柯西(**Cauchy**)分布等，从而构成玻尔兹曼机、高斯机、柯西机等随机机。

8.4.3 随机神经网络

1. Boltzmann机

- 1985年，加拿大多伦多大学教授欣顿(Hinton)等人借助统计物理学的概念和方法，提出了Boltzmann机神经网络模型。
- Boltzmann机是离散Hopfield神经网络的一种变型，通过对离散Hopfield神经网络加以扰动，使其以概率的形式表达，而网络的模型方程不变，只是输出值类似于Boltzmann分布以概率分布取值。
- Boltzmann机是按Boltzmann概率分布动作的神经网络。

8.4.3 随机神经网络

1. Boltzmann机（续）

□ 离散Hopfield神经网络的输出：

$$v_i(t+1) = \text{sgn}\left(\sum_{\substack{j=1 \\ j \neq i}}^n w_{ij} v_j(t) - \theta_i\right)$$

□ Boltzman机的内部状态：

$$I_i = \sum_{\substack{j=1 \\ j \neq i}}^n w_{ij} v_j(t) - \theta_i$$

□ 神经元 i 输出值为0和1时的概率：

$$p_i(1) = \frac{1}{1 + e^{-I_i/T}} \qquad p_i(0) = 1 - p_i(1)$$

8.4.3 随机神经网络

1. Boltzmann机（续）

□ Boltzmann的能量函数：

$$E = -\frac{1}{2} \sum_i \sum_{j \neq i} w_{ij} v_i v_j - \sum_i v_i \theta_i$$

- 神经元 i 状态转换时网络能量的变化：

$$\Delta E_i = \sum_j w_{ij} v_j + \theta_i$$

- 神经元 i 改变为状态“1”的概率：

$$p_i = \frac{1}{1 + \exp(-\frac{\Delta E_i}{T})}$$

8.4.3 随机神经网络

2. 高斯机

$$\frac{du_i}{dt} = \sum_{j=1}^n w_{ij} v_j + I_i + \eta$$

η : 均值为0的高斯随机变量（白噪声），其方差为 $\sigma = cT$

3. 柯西机

$$\frac{du_i}{dt} = \sum_{j=1}^n w_{ij} v_j + I_i + \eta$$

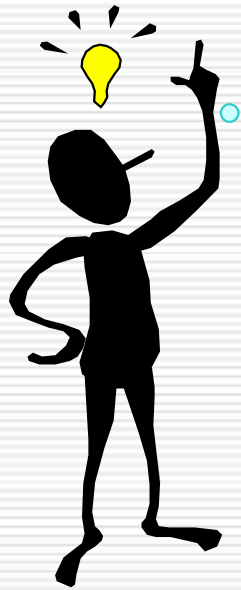
η : 柯西随机变量（有色噪声）

8.5 Hopfield神经网络的应用

- **8.5.1 Hopfield神经网络在联想记忆中的应用**
- **8.5.2 Hopfield神经网络优化方法**

8.5.1 Hopfield神经网络在联想记忆中的应用

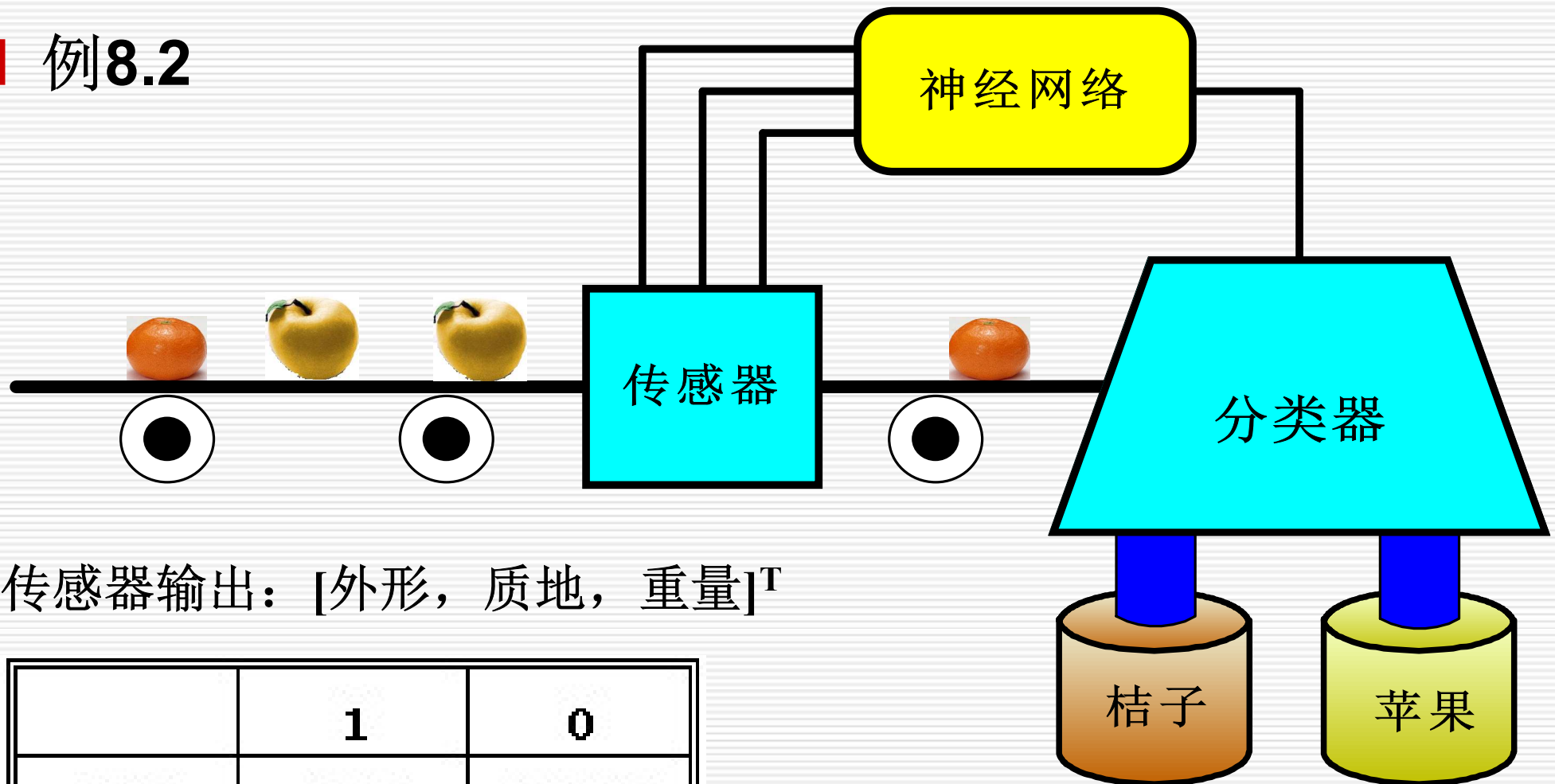
如何实现HNN的联想记忆功能？



- 网络能够通过联想来输出和输入模式最为相似的样本模式。

8.5.1 Hopfield神经网络在联想记忆中的应用

□ 例8.2



■ 传感器输出：[外形，质地，重量]^T

	1	0
外形	圆形	椭圆形
质地	光滑	粗糙
重量	< 1 磅	> 1 磅

标准桔子： $x^{(1)} = [1 \ 0 \ 1]^T$

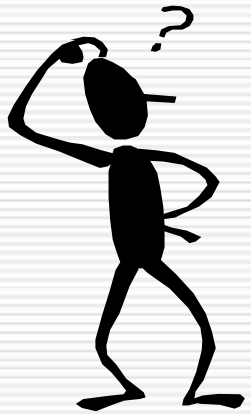
标准苹果： $x^{(2)} = [0 \ 1 \ 0]^T$

8.5.1 Hopfield神经网络在联想记忆中的应用

□ 例8.2

■ 样本: $x^{(1)} = [1, 0, 1]^T$ (桔子) $x^{(2)} = [0, 1, 0]^T$ (苹果)

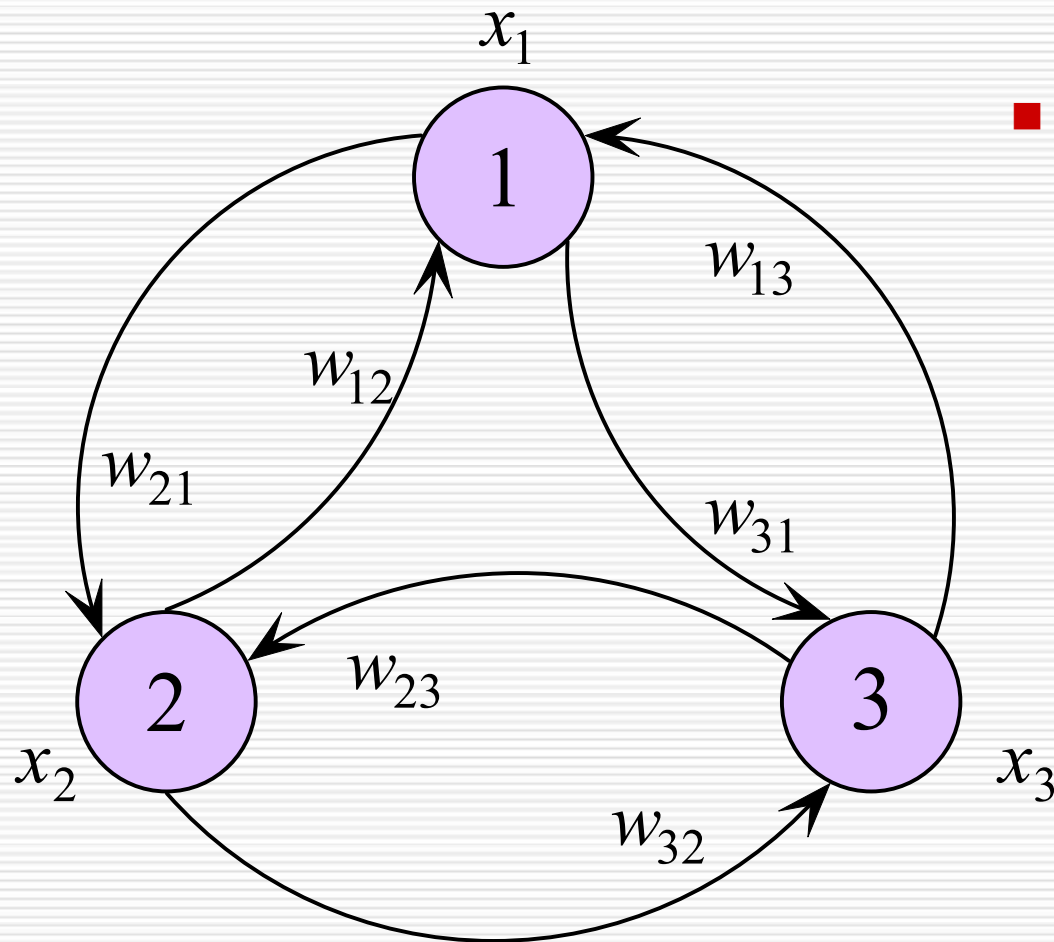
具体怎样实现联想
记忆?



- 步骤:
 - (1) 设计**DHNN**结构
 - (2) 设计连接权矩阵
 - (3) 测试

8.5.1 Hopfield神经网络在联想记忆中的应用

■ (1) 设计DHNN结构



- 样本: $x^{(1)} = [1, 0, 1]^T$ (桔子)
 $x^{(2)} = [0, 1, 0]^T$ (苹果)

注: $\theta_i = 0 (i = 1, 2, 3)$

3神经元的DHNN结构图

8.5.1 Hopfield神经网络在联想记忆中的应用

■ (2) 设计连接权矩阵

■ 样本: $x^{(1)} = [1, 0, 1]^T$ (桔子) $x^{(2)} = [0, 1, 0]^T$ (苹果)

■ 连接权: $w_{ij} = \begin{cases} \sum_{l=1}^2 (2x_i^{(l)} - 1)(2x_j^{(l)} - 1) & i \neq j \\ 0 & i = j \end{cases}$

$$w_{ji} = w_{ij} \quad (i = 1, 2, \dots, n; j = 1, 2, \dots, n)$$

$$w_{12} = (2x_1^{(1)} - 1)(2x_2^{(1)} - 1) + (2x_1^{(2)} - 1)(2x_2^{(2)} - 1)$$

$$= (2 \times 1 - 1) \times (2 \times 0 - 1) + (2 \times 0 - 1) \times (2 \times 1 - 1)$$

$$= -1 - 1 = -2$$

$$w_{21} = w_{12} = -2$$

8.5.1 Hopfield神经网络在联想记忆中的应用

■ (2) 设计连接权矩阵

■ 样本: $x^{(1)} = [1, 0, 1]^T$ $x^{(2)} = [0, 1, 0]^T$

■ 连接权: $w_{ij} = \begin{cases} \sum_{l=1}^2 (2x_i^{(l)} - 1)(2x_j^{(l)} - 1) & i \neq j \\ 0 & i = j \end{cases}$

$$w_{ji} = w_{ij} (i = 1, 2, \dots, n; j = 1, 2, \dots, n)$$

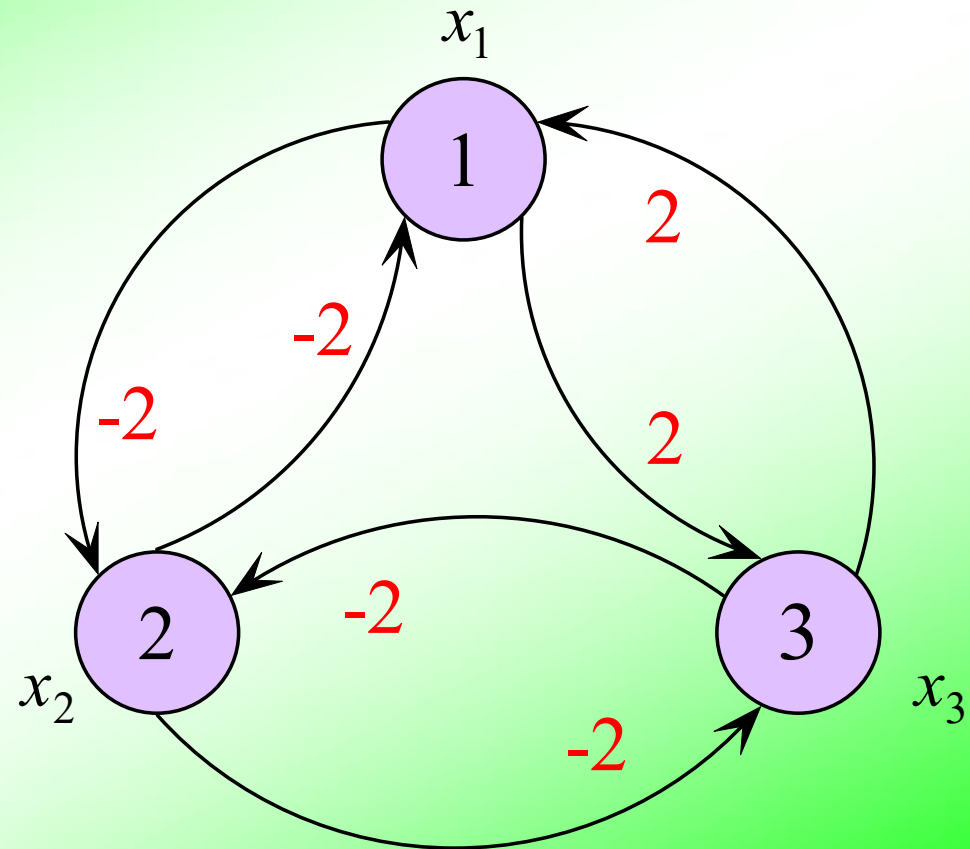
$$\begin{aligned} w_{23} &= (2x_2^{(1)} - 1)(2x_3^{(1)} - 1) + (2x_2^{(2)} - 1)(2x_3^{(2)} - 1) \\ &= (2 \times 0 - 1) \times (2 \times 1 - 1) + (2 \times 1 - 1) \times (2 \times 0 - 1) \\ &= -1 + (-1) = -2 \end{aligned}$$

$$w_{32} = w_{23} = -2$$

8.5.1 Hopfield神经网络在联想记忆中的应用

■ (2) 设计连接权矩阵

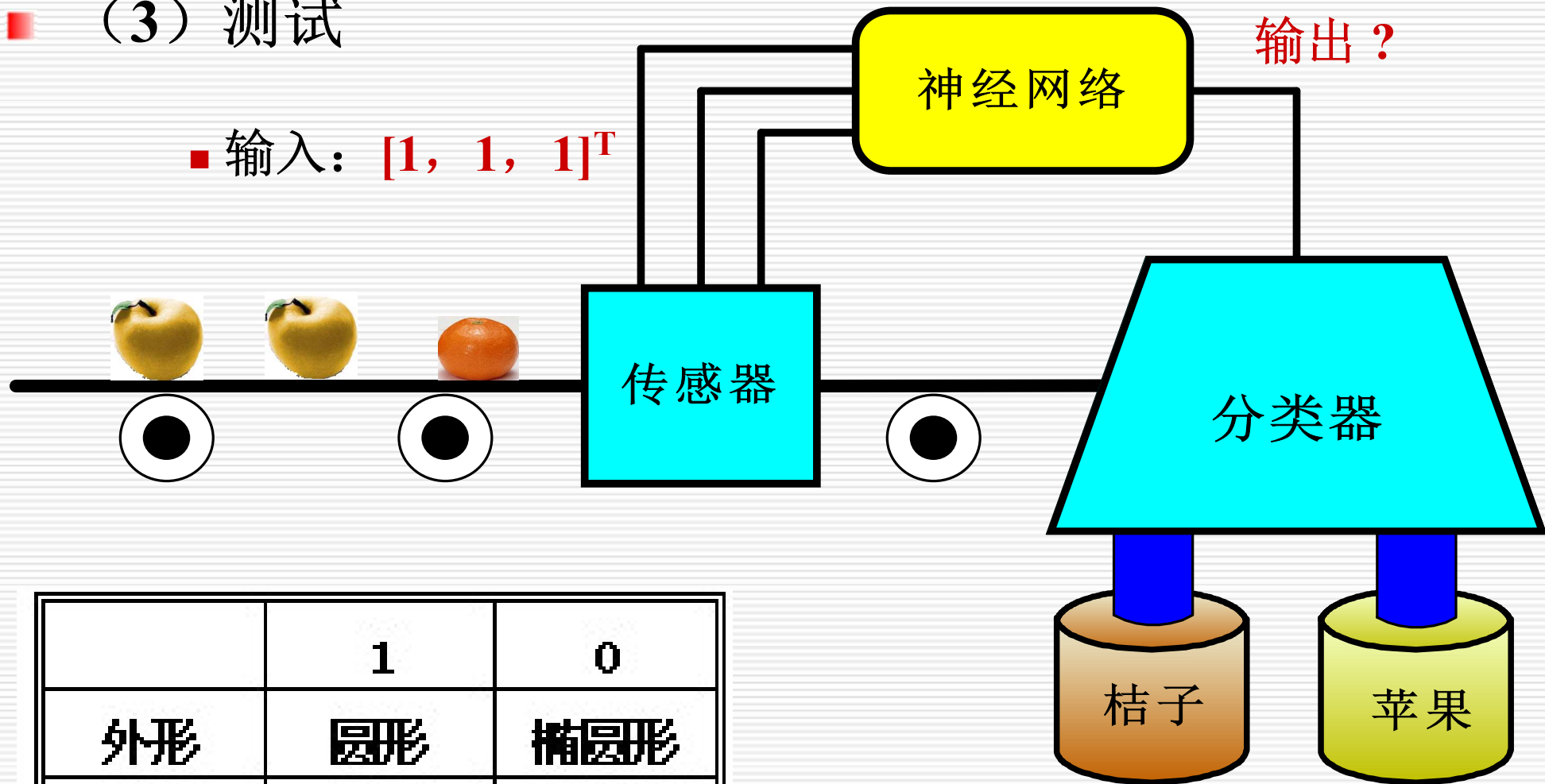
$$W = \begin{bmatrix} 0 & -2 & 2 \\ -2 & 0 & -2 \\ 2 & -2 & 0 \end{bmatrix}$$



8.5.1 Hopfield神经网络在联想记忆中的应用

■ (3) 测试

■ 输入: $[1, 1, 1]^T$



	1	0
外形	圆形	椭圆形
质地	光滑	粗糙
重量	< 1 磅	> 1 磅

标准桔子: $x^{(1)} = [1, 0, 1]^T$

标准苹果: $x^{(2)} = [0, 1, 0]^T$

8.5.1 Hopfield神经网络在联想记忆中的应用

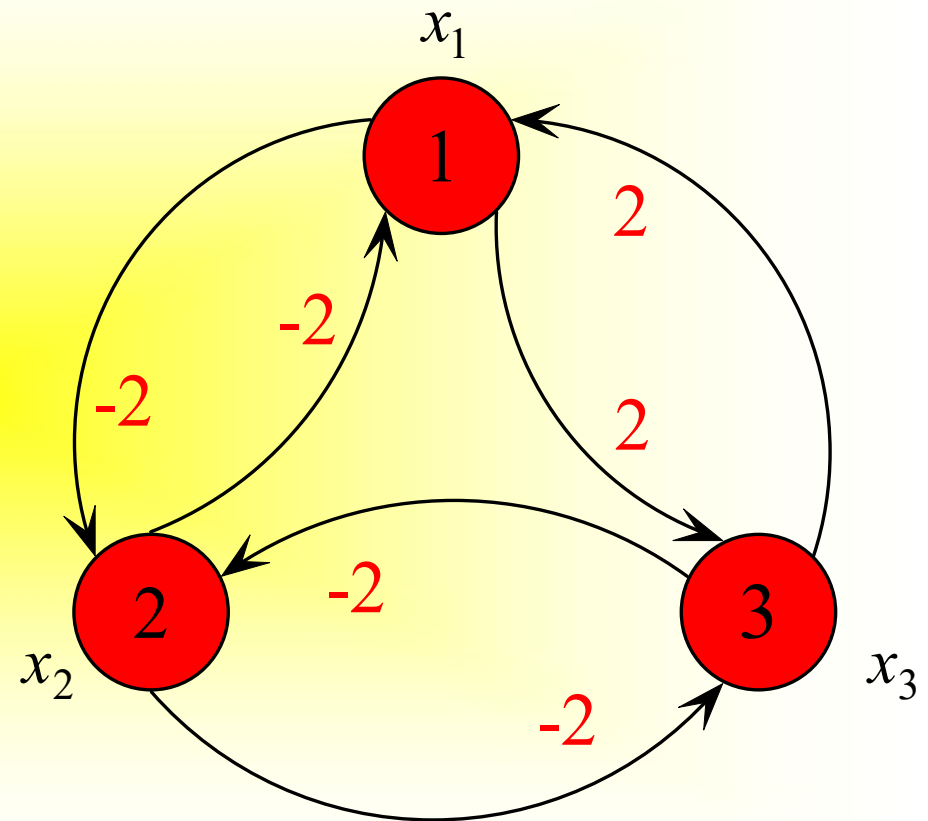
■ (3) 测试

- 样本: $x^{(1)} = [1, 0, 1]^T$ (桔子) $x^{(2)} = [0, 1, 0]^T$ (苹果)

- 测试用例: $[1, 1, 1]^T$

- 初始状态:
$$\begin{cases} x_1(0) = 1 \\ x_2(0) = 1 \\ x_3(0) = 1 \end{cases}$$

- 调整次序: $2 \rightarrow 1 \rightarrow 3$

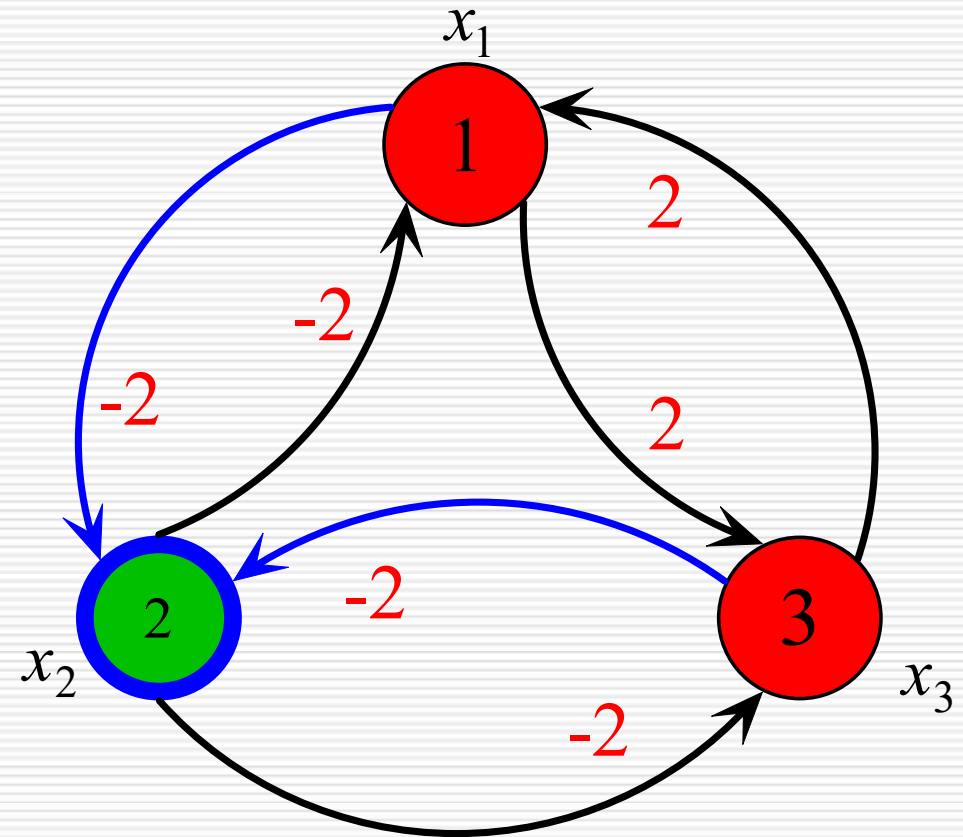


8.5.1 Hopfield神经网络在联想记忆中的应用

- 调整次序: **2**→1→3

$k = 0$

$$\begin{cases} x_1(0) = 1 \\ x_2(0) = 1 \\ x_3(0) = 1 \end{cases}$$



$$x_2(1) = f\left(\sum_{j=1}^3 w_{2j}x_j(0)\right) = f((-2) \times 1 + 0 \times 1 + (-2) \times 1) = f(-4) = 0$$

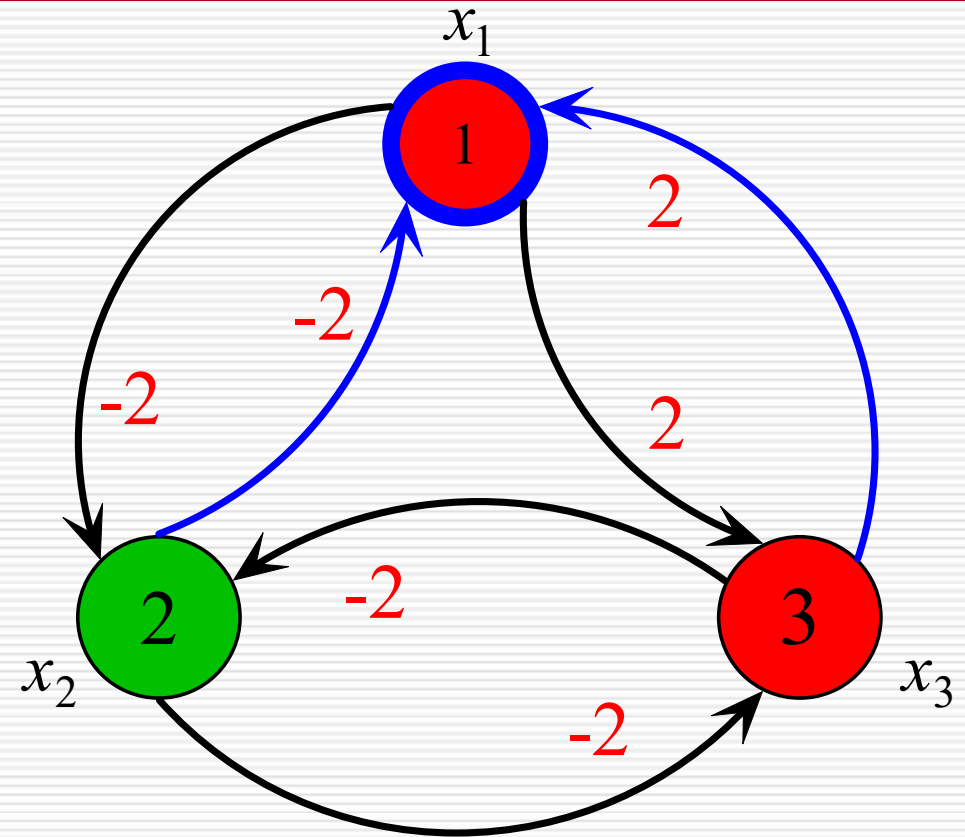
$$x_1(1) = x_1(0) = 1, \quad x_3(1) = x_3(0) = 1$$

8.5.1 Hopfield神经网络在联想记忆中的应用

- 调整次序: $2 \rightarrow \mathbf{1} \rightarrow 3$

$$k = 1$$

$$\begin{cases} x_1(1) = 1 \\ x_2(1) = 0 \\ x_3(1) = 1 \end{cases}$$



$$x_1(2) = f\left(\sum_{j=1}^3 w_{1j}x_j(1)\right) = f(0 \times 1 + (-2) \times 0 + 2 \times 1) = f(2) = 1$$

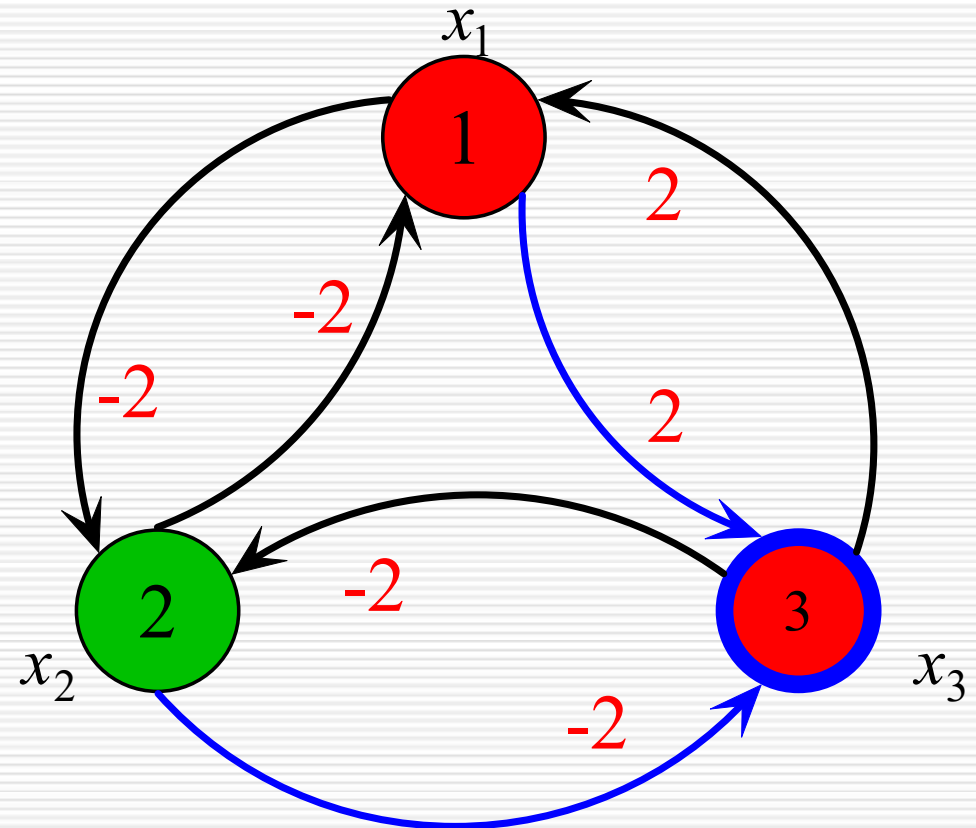
$$x_2(2) = x_2(1) = 0, \quad x_3(2) = x_3(1) = 1$$

8.5.1 Hopfield神经网络在联想记忆中的应用

- 调整次序: **2**→**1**→**3**

$$k = 2$$

$$\begin{cases} x_1(2) = 1 \\ x_2(2) = 0 \\ x_3(2) = 1 \end{cases}$$



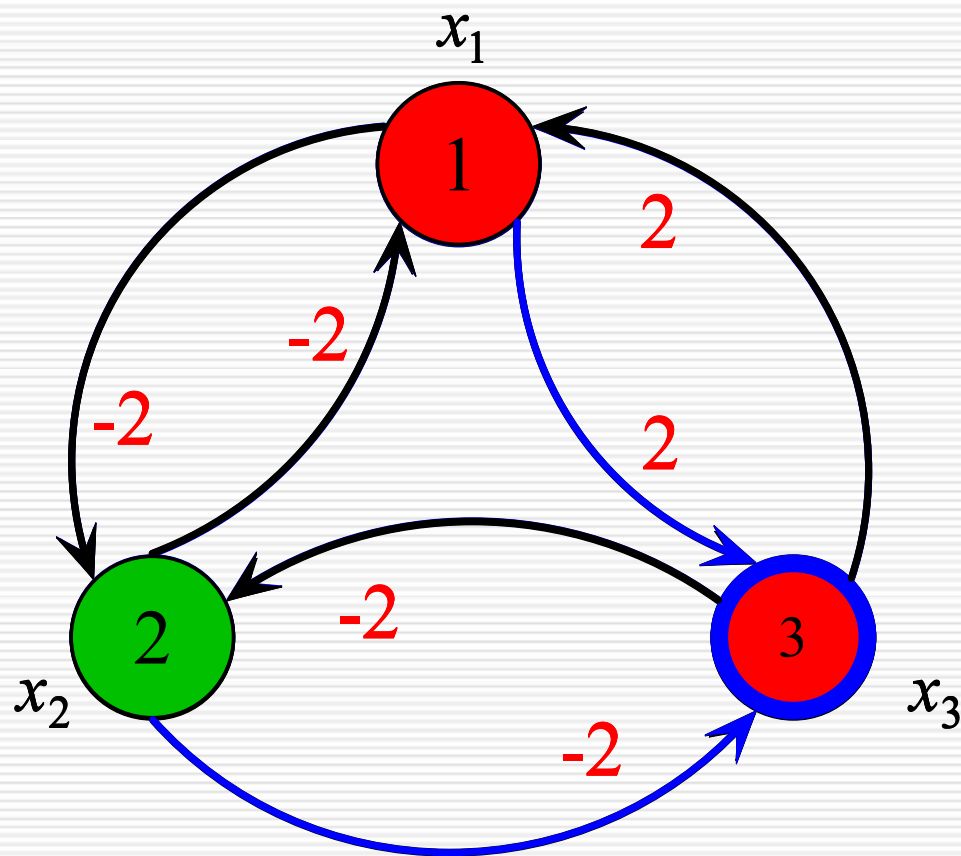
$$x_3(3) = f\left(\sum_{j=1}^3 w_{3j}x_j(1)\right) = f(2 \times 1 + (-2) \times 0 + 0 \times 1) = f(2) = 1$$

$$x_1(3) = x_1(2) = 1, \quad x_2(3) = x_2(2) = 0$$

8.5.1 Hopfield神经网络在联想记忆中的应用

- 调整次序: **2** $\rightarrow$ **1** $\rightarrow$ **3**
- 样本: $x^{(1)} = [1, 0, 1]^T$ (桔子)

$$x^{(2)} = [0, 1, 0]^T \text{ (苹果)}$$



$$k = 0$$

$$\begin{cases} x_1(0) = 1 \\ x_2(0) = 1 \\ x_3(0) = 1 \end{cases}$$

$$k = 1$$

$$\begin{cases} x_1(1) = 1 \\ x_2(1) = 0 \\ x_3(1) = 1 \end{cases}$$

$$k = 2$$

$$\begin{cases} x_1(2) = 1 \\ x_2(2) = 0 \\ x_3(2) = 1 \end{cases}$$

$$k = 3$$

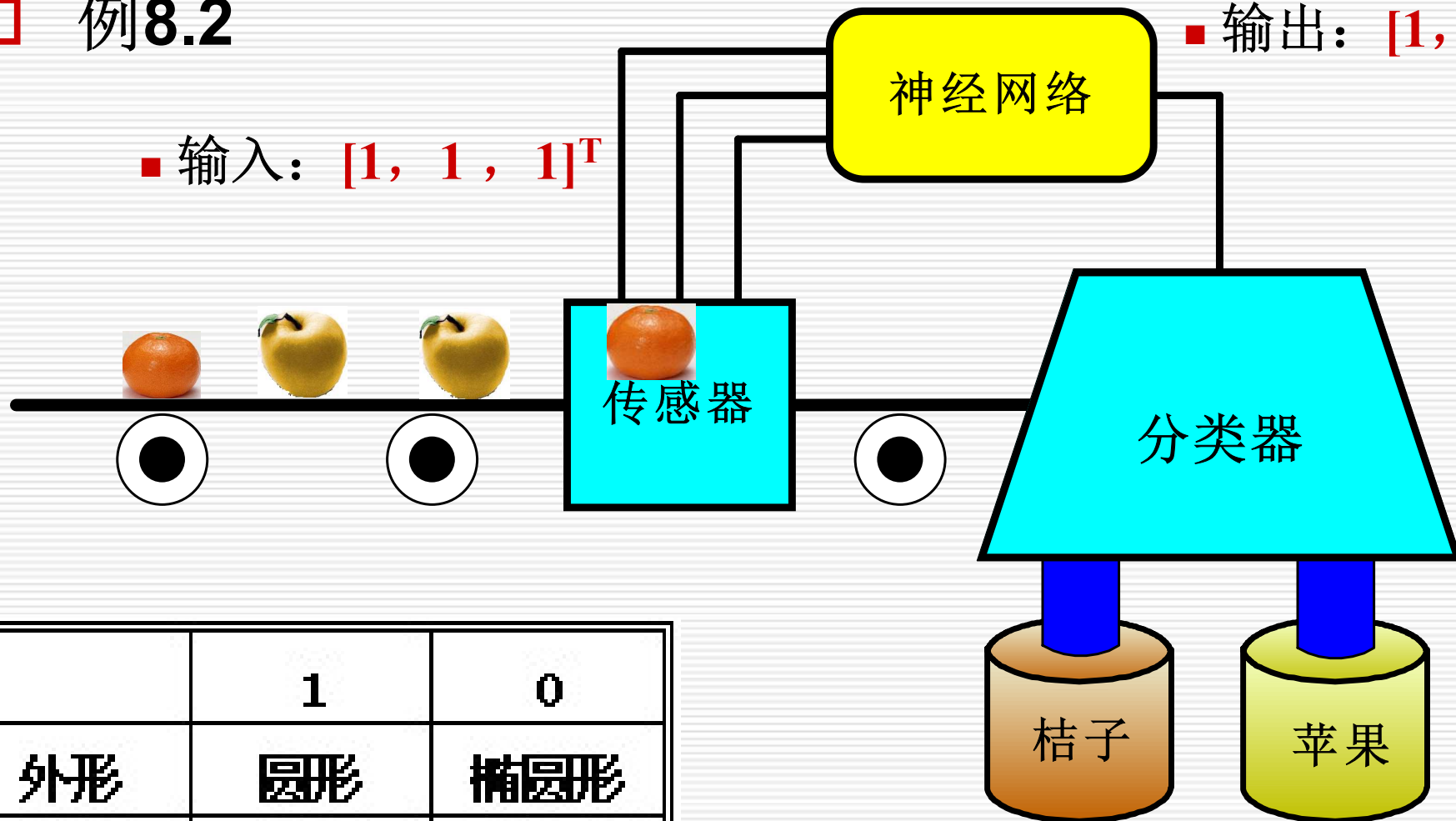
$$\begin{cases} x_1(3) = 1 \\ x_2(3) = 0 \\ x_3(3) = 1 \end{cases}$$

8.5.1 Hopfield神经网络在联想记忆中的应用

□ 例8.2

■ 输入: $[1, 1, 1]^T$

■ 输出: $[1, 0, 1]^T$



	1	0
外形	圆形	椭圆形
质地	光滑	粗糙
重量	< 1 磅	> 1 磅

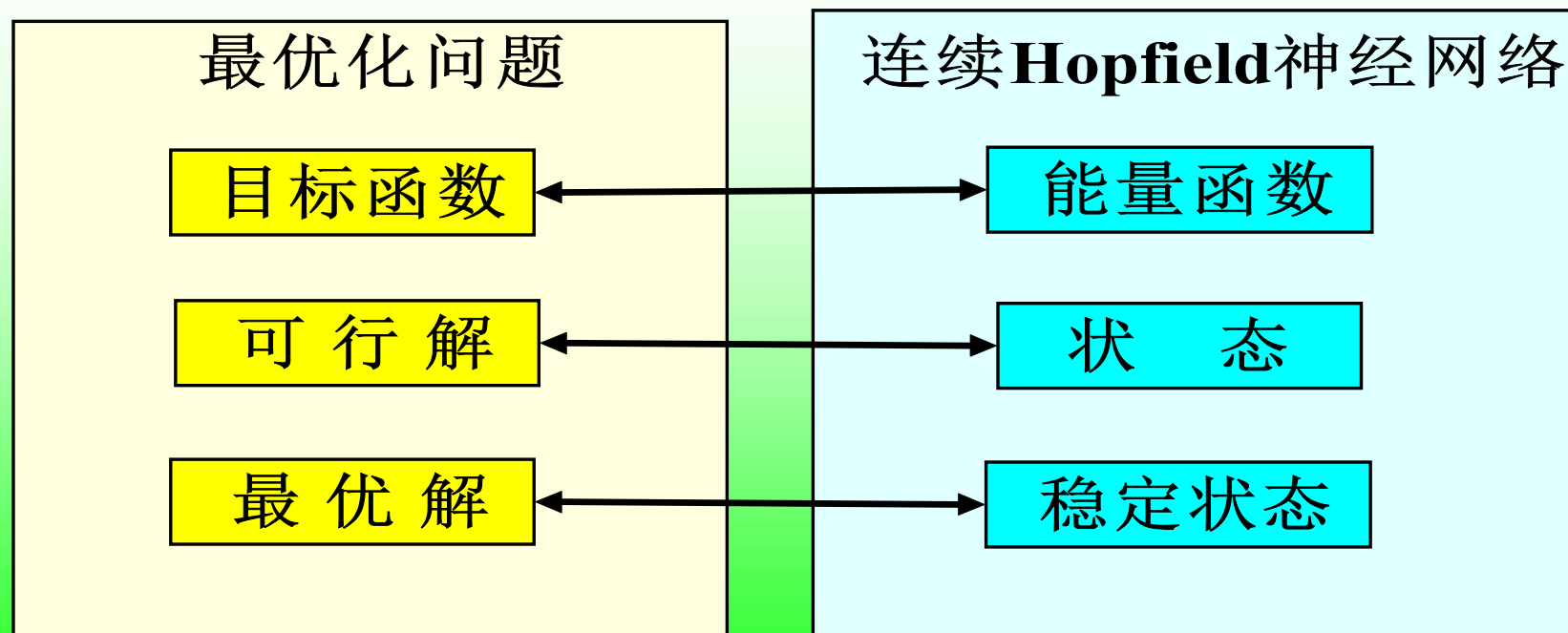
标准桔子: $x^{(1)} = [1, 0, 1]^T$

标准苹果: $x^{(2)} = [0, 1, 0]^T$

8.5.2 Hopfield神经网络优化方法

- 1985年，霍普菲尔德和塔克（D. W. Tank）应用连续Hopfield神经网络求解旅行商问题（traveling salesman problem, TSP）获得成功。

- 连续Hopfield神经网络求解约束优化问题的基本思路：



8.5.2 Hopfield神经网络优化方法

□ 用神经网络方法求解优化问题的一般步骤：

- (1) 将优化问题的每一个可行解用换位矩阵表示。
- (2) 将换位矩阵与由 n 个神经元构成的神经网络相对应：每一个可行解的换位矩阵的各元素与相应的神经元稳态输出相对应。
- (3) 构造能量函数，使其最小值对应于优化问题的最优解，并满足约束条件。
- (4) 用罚函数法构造目标函数，与Hopfield神经网络的计算能量函数表达式相等，确定各连接权和偏置参数。
- (5) 给定网络初始状态和网络参数等，使网络按动态方程运行，直到稳定状态，并将它解释为优化问题的解。

8.5.2 Hopfield神经网络优化方法

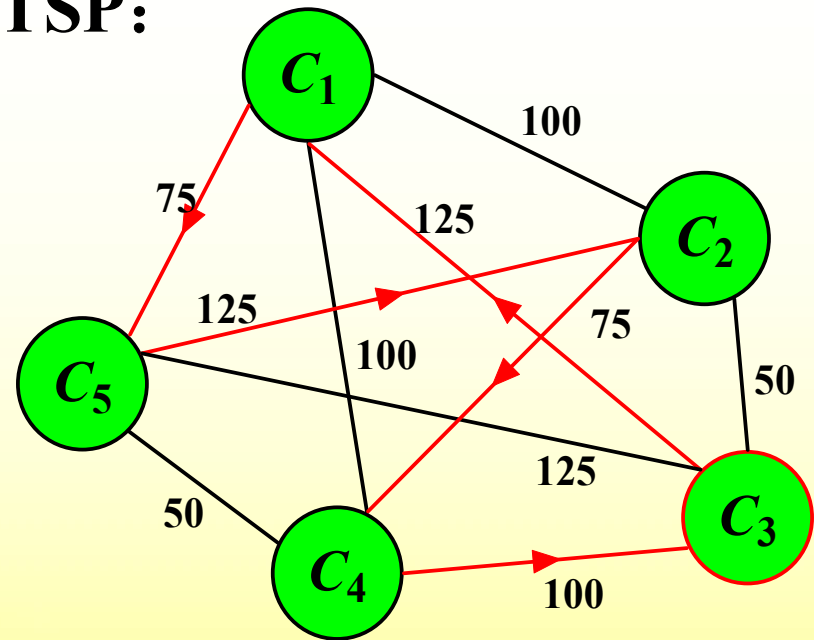
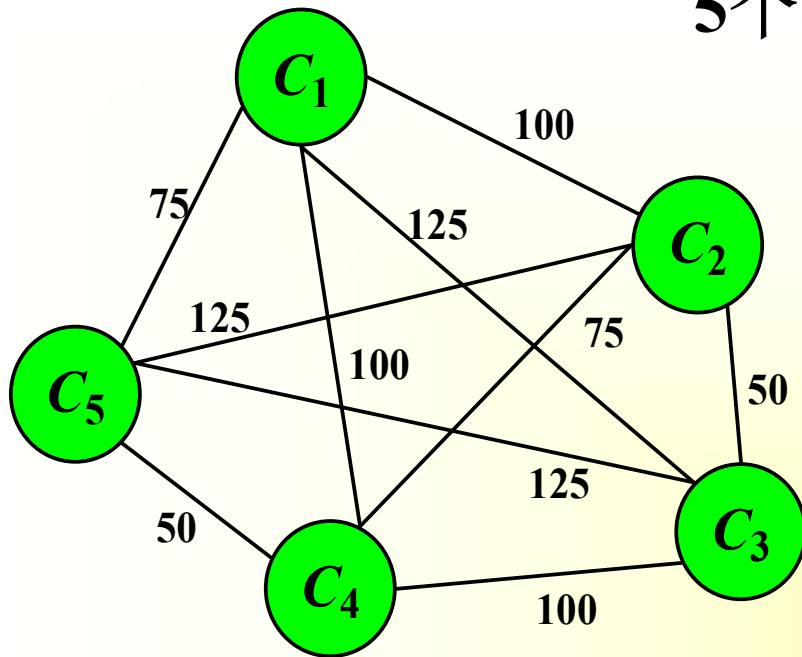
□ 应用举例：Hopfield神经网络优化方法求解TSP。

■ 1985年，霍普菲尔德和塔克（D. W. Tank）应用连续Hopfield神经网络求解旅行商问题获得成功。

■ 旅行商问题（traveling salesman problem, TSP）：有 n 个城市，城市间的距离或旅行成本已知，求合理的路线使每个城市都访问一次，且总路径（或者总成本）为最短。

8.5.2 Hopfield神经网络优化方法

5个城市的TSP:



	1	2	3	4	5
C_1	0	1	0	0	0
C_2	0	0	0	1	0
C_3	1	0	0	0	0
C_4	0	0	0	0	1
C_5	0	0	1	0	0

神经元数目: 25

8.5.2 Hopfield神经网络优化方法

■ TSP的描述:

$$\min l = \frac{1}{2} \sum_x \sum_{y \neq x} \sum_i d_{xy} v_{xi} (v_{y,i+1} + v_{y,i-1})$$

$$st. \quad \sum_x v_{xi} = 1 \quad \forall i \quad (\text{每次只能访问一个城市})$$

$$\sum_i v_{xi} = 1 \quad \forall x \quad (\text{每个城市只能访问一次})$$

■ 用罚函数法，写出优化问题的目标函数:

$$J = \frac{A}{2} \sum_x \sum_i \sum_{j \neq i} v_{xi} v_{xj} + \frac{B}{2} \sum_i \sum_x \sum_{y \neq x} v_{xj} v_{yi} + \frac{C}{2} (\sum_x \sum_i v_{xi} - n)^2 +$$
$$\frac{D}{2} \sum_x \sum_{y \neq x} \sum_i d_{xy} v_{xi} (v_{y,i+1} + v_{y,i-1})$$

目标约束

8.5.2 Hopfield神经网络优化方法

- Hopfield神经网络能量函数：

$$\begin{aligned} E &= -\frac{1}{2} \sum_{x=1}^n \sum_{i=1}^n \sum_{y=1}^n \sum_{j=1}^n w_{xi,yj} v_{xi} v_{yj} - \sum_{x=1}^n \sum_{i=1}^n v_{xi} I_{xi} + \sum_{x=1}^n \sum_{i=1}^n \frac{1}{R'_{xi}} \int_0^{v_{xi}} f^{-1}(v) dv \\ &= E_1 + \sum_{x=1}^n \sum_{i=1}^n \frac{1}{R'_{xi}} \int_0^{v_{xi}} f^{-1}(v) dv \end{aligned}$$

- 令 E_1 与目标函数 J 相等，确定神经网络的连接权值和偏置电流：

$$W_{xi,yj} = A\delta_{xy}(1-\delta_{ij}) - B\delta_{ij}(1-\delta_{xy}) - C - D\delta_{xy}(\delta_{j,i+1} + \delta_{j,i-1})$$

$$I_{xi} = -Cn$$

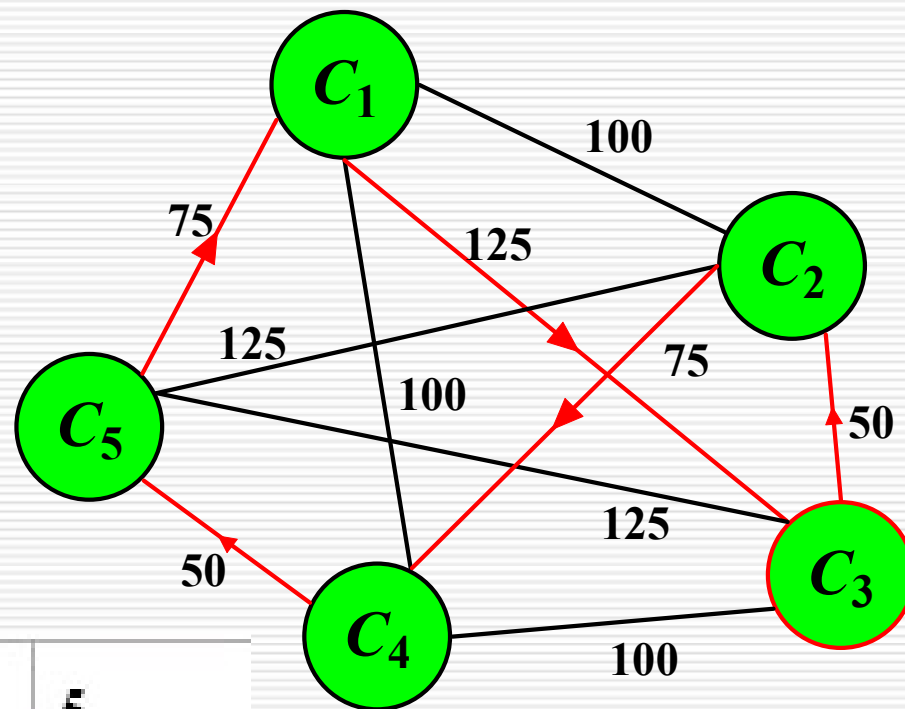
8.5.2 Hopfield神经网络优化方法

■ 神经网络的动态方程:

$$\begin{aligned} C_{xi} \frac{du_{xj}}{dt} &= -\frac{\partial E}{\partial v_{xi}} = -\frac{\partial(E_1 + \sum_{x=1}^n \sum_{i=1}^n \frac{1}{R'_{xi}} \int_0^{v_{xi}} f^{-1}(v) dv)}{\partial v_{xi}} \\ &= -\frac{\partial(J + \sum_{x=1}^n \sum_{i=1}^n \frac{1}{R'_{xi}} \int_0^{v_{xi}} f^{-1}(v) dv)}{\partial v_{xi}} \\ &= -\frac{u_{xi}}{R'_{xi}} - A \sum_{j \neq i} v_{xj} - B \sum_{y \neq x} v_{yi} - C(\sum_x \sum_i v_{xi} - n) - D \sum_y d_{xy} (v_{y,i+1} + v_{y,i-1}) \\ v_{xi} &= f(u_{xi}) = \frac{1}{2} [1 + \tanh(\frac{u_{xi}}{u_0})] \end{aligned}$$

8.5.2 Hopfield神经网络优化方法

- 选择合适的 A 、 B 、 C 、 D 和网络的初始状态，按网络动态方程演化直到收敛。



	1	2	3	4	5
C_1	0	0	0	0	1
C_2	0	1	0	0	0
C_3	1	0	0	0	0
C_4	0	0	1	0	0
C_5	0	0	0	1	0

8.5.2 Hopfield神经网络优化方法

- 神经网络优化计算目前存在的问题：
 - (1) 解的不稳定性。
 - (2) 参数难以确定。
 - (3) 能量函数存在大量局部极小值，难以保证最优解。

第8章 人工神经网络及其应用

- 8.1 神经元与神经网络
- 8.2 BP神经网络及其学习算法
- 8.3 BP神经网络在模式识别中的应用
- 8.4 Hopfield神经网络及其改进
- 8.5 Hopfield神经网络的应用
- 8.6 卷积神经网络与深度学习
- 8.7 生成对抗网络及其应用

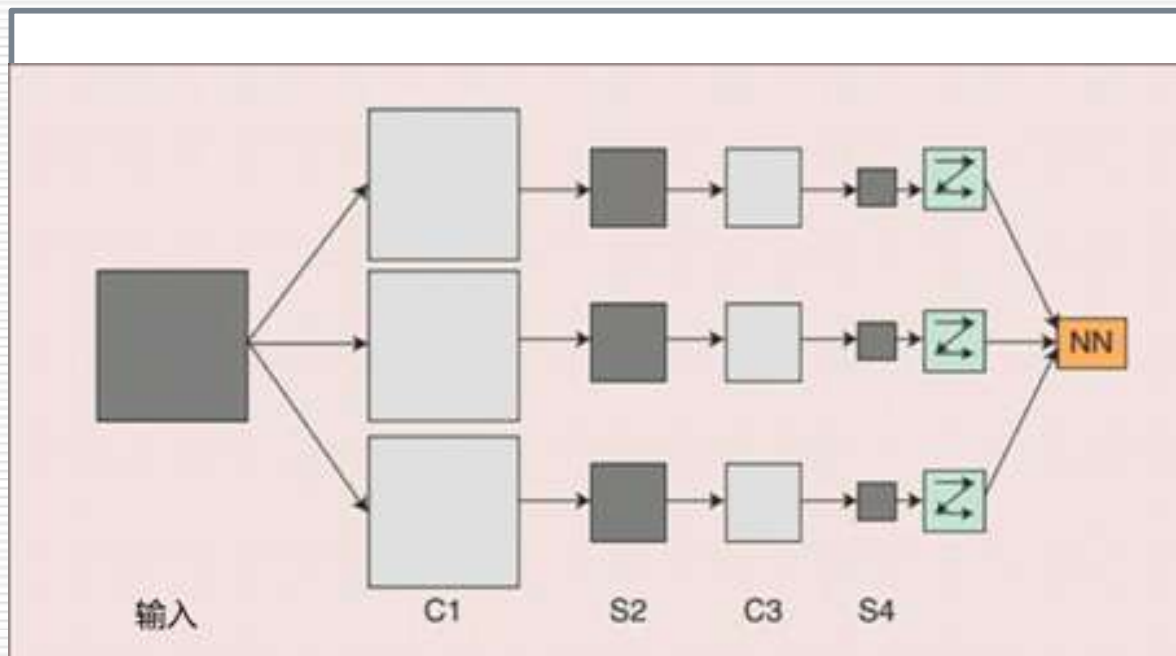
8.6 卷积神经网络与深度学习

□ 卷积神经网络（Convolutional Neural Networks, CNN）

- 1962年Hubel和Wiesel通过对猫视觉皮层细胞的研究，提出了感受野(receptive field)的概念。视觉皮层的神经元就是局部接受信息的，只受某些特定区域刺激的响应，而不是对全局图像进行感知。
- 1984年日本学者Fukushima基于感受野概念提出神经认知机(neocognitron)，被认为是最早的“卷积神经网络”（或说 CNN 的雏形）。
- CNN可看作是神经认知机的推广形式。

8.6.1 卷积神经网络的结构

□ 卷积神经网络的结构

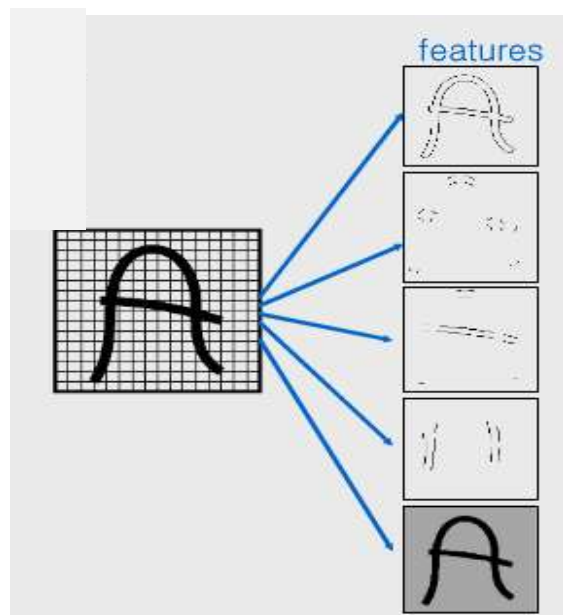
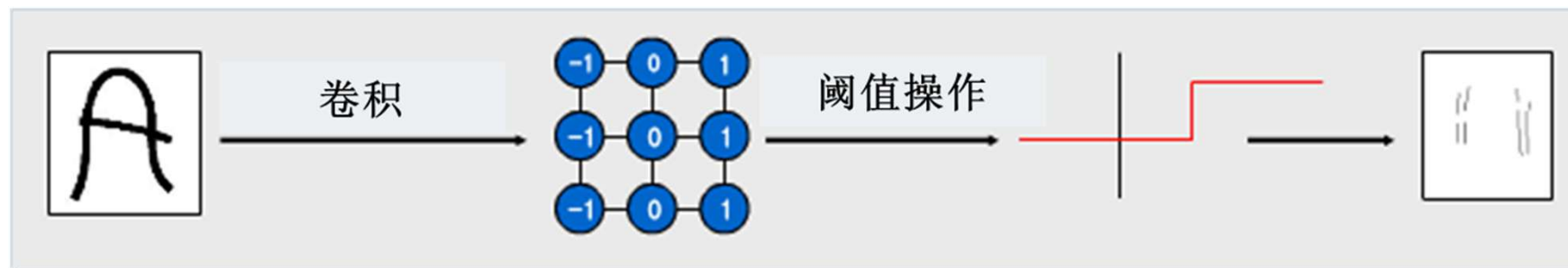


概念示范：输入图像通过与 m 个可训练的滤波器和可加偏置进行卷积，在**C1**层产生 m 个特征映射图，然后特征映射图中每组的 n 个像素再求和，加权值，加偏置，通过**Sigmoid**函数得到 m 个**S2**层的特征映射图。这些映射图再经过滤波得到**C3**层。这个层级结构再和**S2**一样产生**S4**。最终，这些像素值被光栅化，并连接成一个向量输入到传统神经网络，得到输出。

- **CNN**是一个多层的神经网络，每层由多个二维平面组成，而每个平面由多个独立神经元组成。
- **C**层为特征提取层（卷积层）
- **S**层是特征映射层（下采样层）。
- **CNN**中的每一个**C**层都紧跟着一个**S**层。

8.6.1 卷积神经网络的结构

□ 特征提取层（卷积层）——C层（**C**onvolution layer）



- 大部分的特征提取都依赖于卷积运算
- 利用卷积算子对图像进行滤波，可以得到显著的边缘特征。

8.6.2 卷积神经网络的卷积运算

1	1	1	0	0
0	1	1	0	1
0	0	1	1	1
0	0	1	1	0
0	1	1	0	0

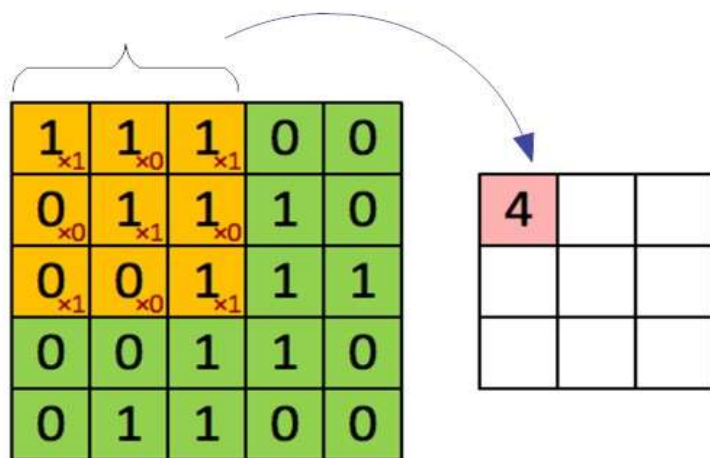
输入数据

*

1	0	1
0	1	0
1	0	1

卷积核

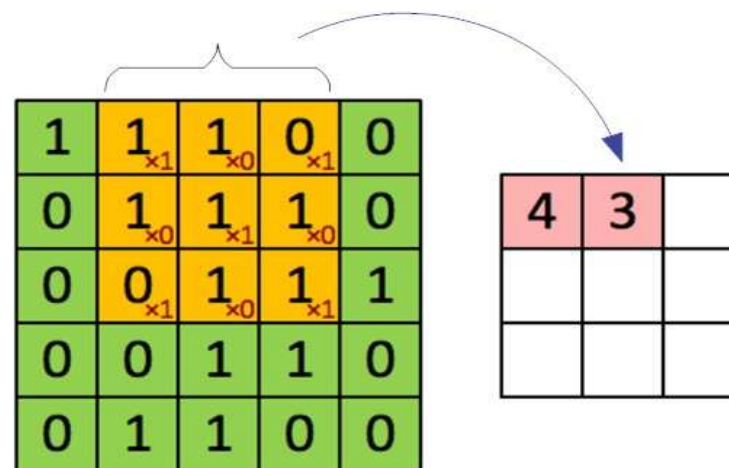
8.6.2 卷积神经网络的卷积运算



图像

卷积特征

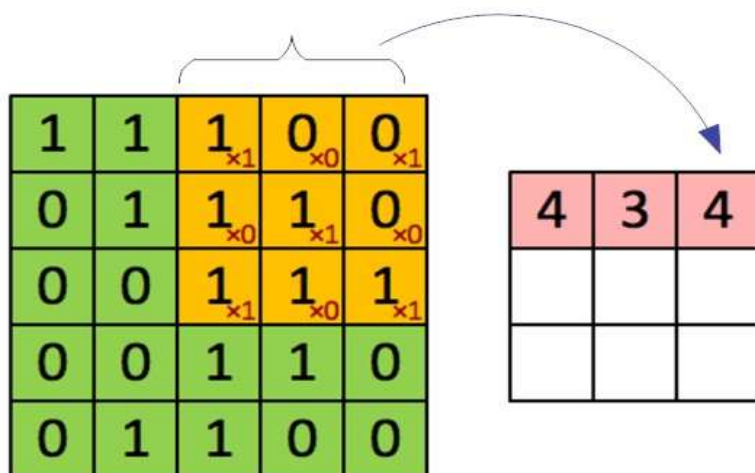
(a)



图像

卷积特征

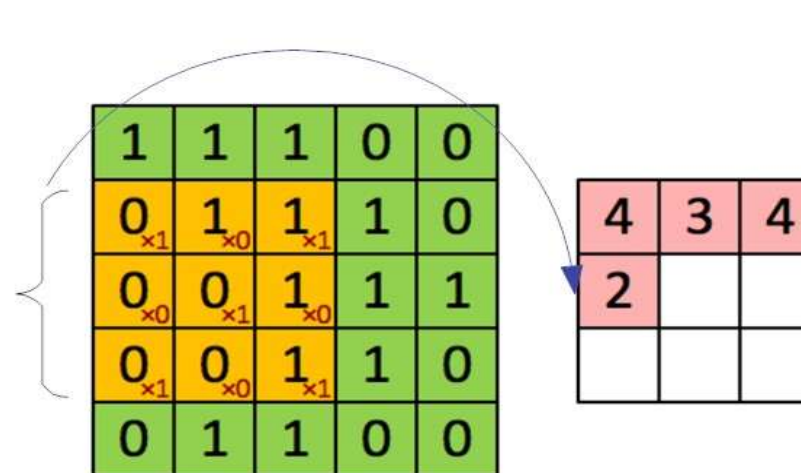
(b)



图像

卷积特征

(c)



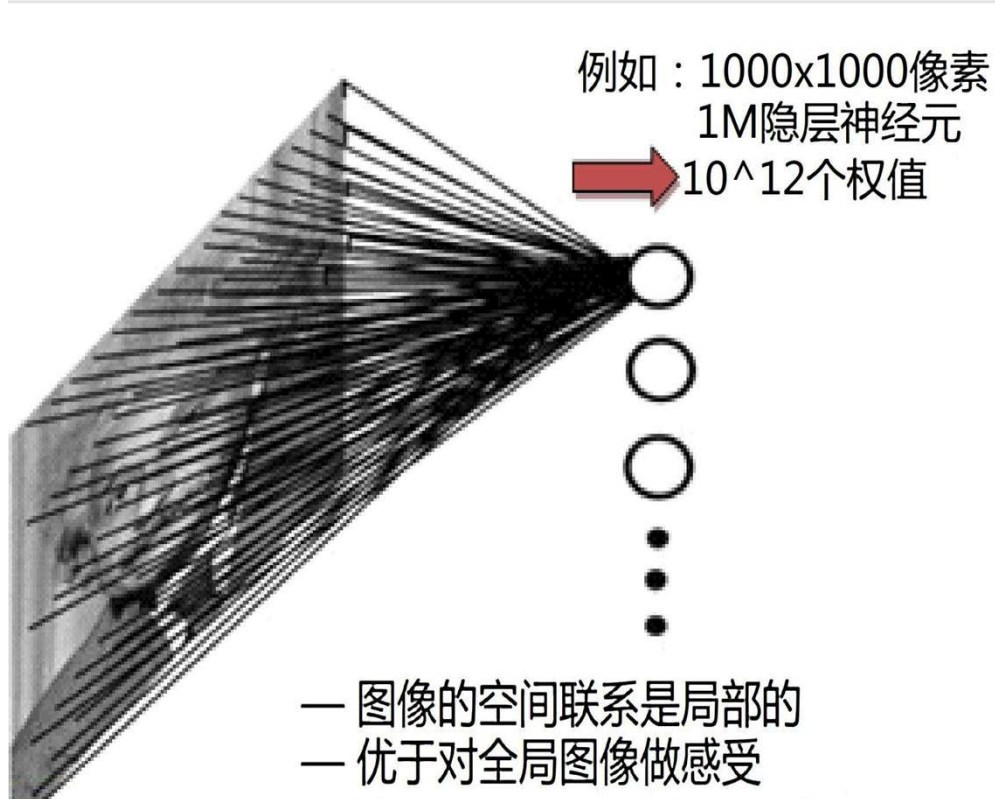
图像

卷积特征

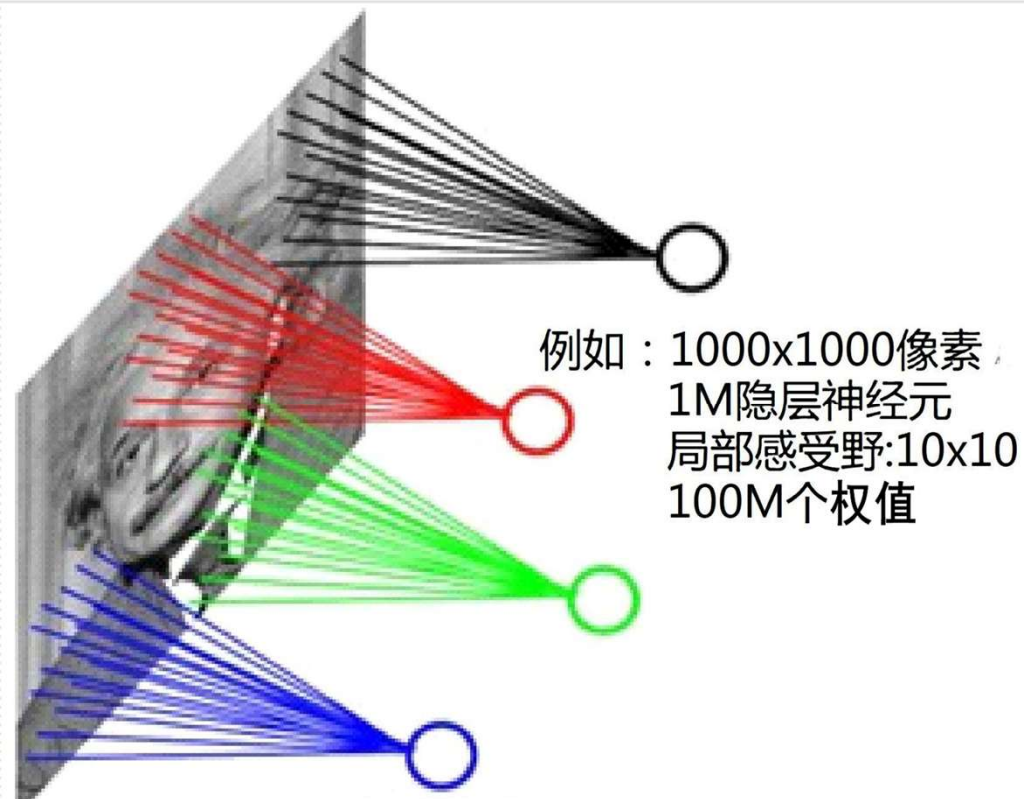
(d)

8.6.3 卷积神经网络中的关键技术

□ 局部链接



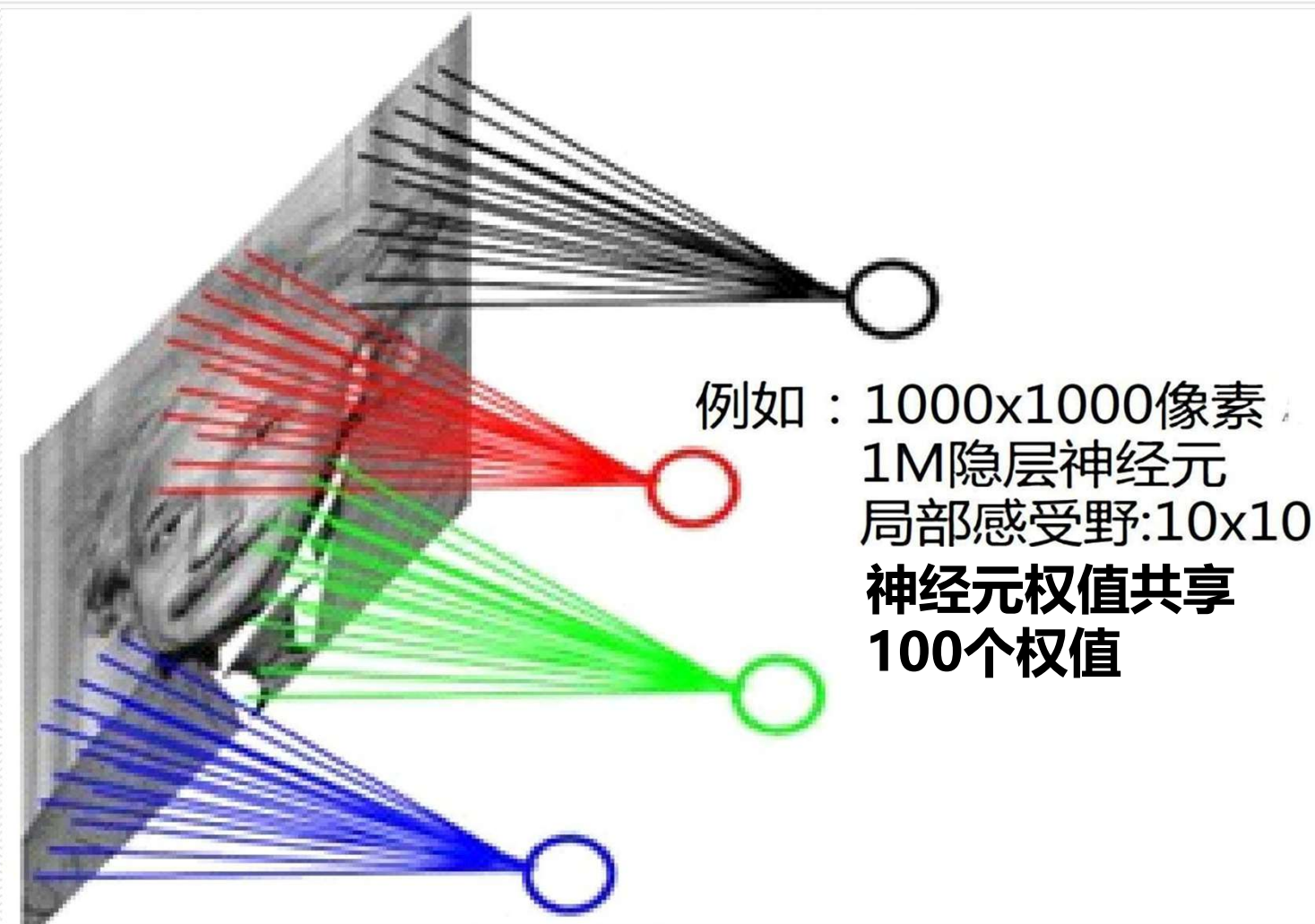
(a) 全连接神经网络



(b) 局部连接神经网络

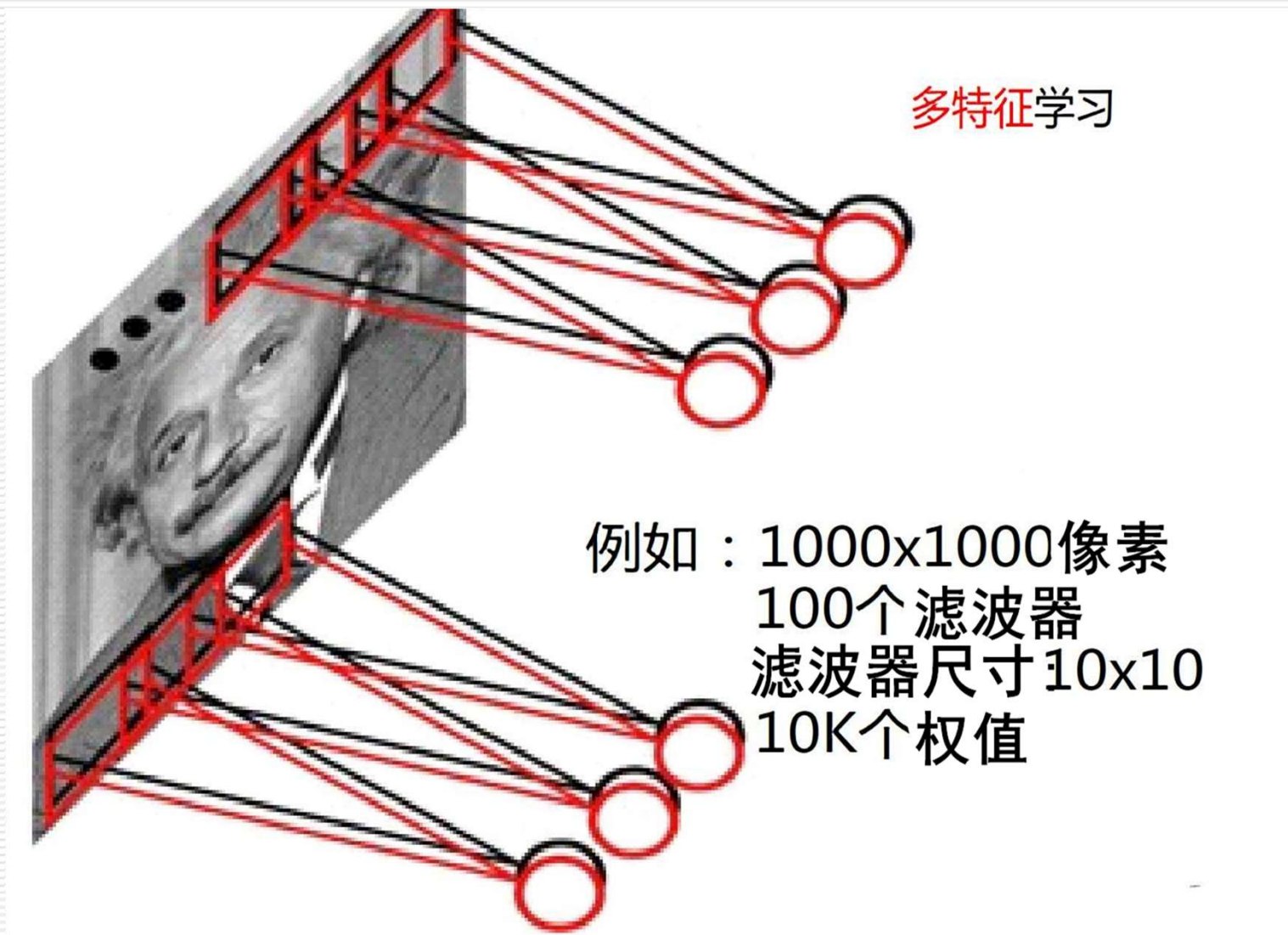
8.6.3 卷积神经网络中的关键技术

□ 权值共享



8.6.3 卷积神经网络中的关键技术

□ 多卷积核



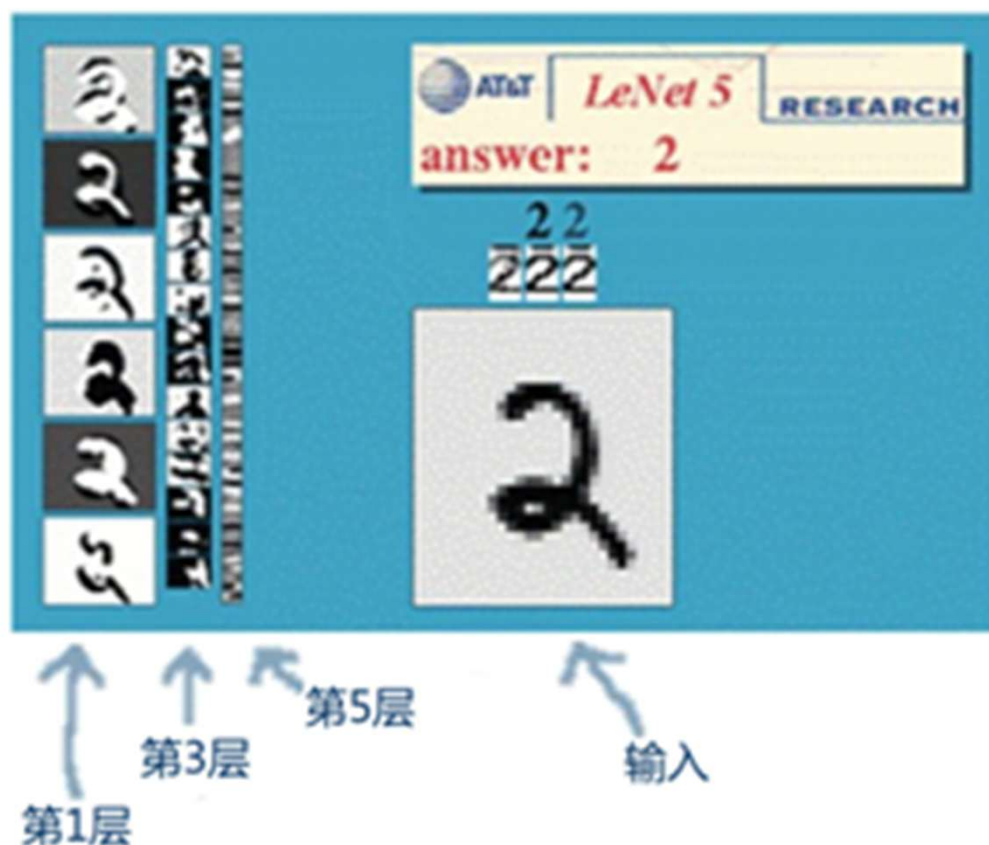
8.6.3 卷积神经网络中的关键技术

□ 池化

- 通过卷积获得了特征之后，如果直接利用这些特征训练分类器，计算量是非常大的。
- 对不同位置的特征进行聚合统计,称为池化 (**pooling**)。
- 池化常用方法：平均池化、最大池化。
- 卷积神经网络在池化层丢失大量的信息，从而降低了空间分辨率，导致了对于输入微小的变化，其输出几乎是不变的。

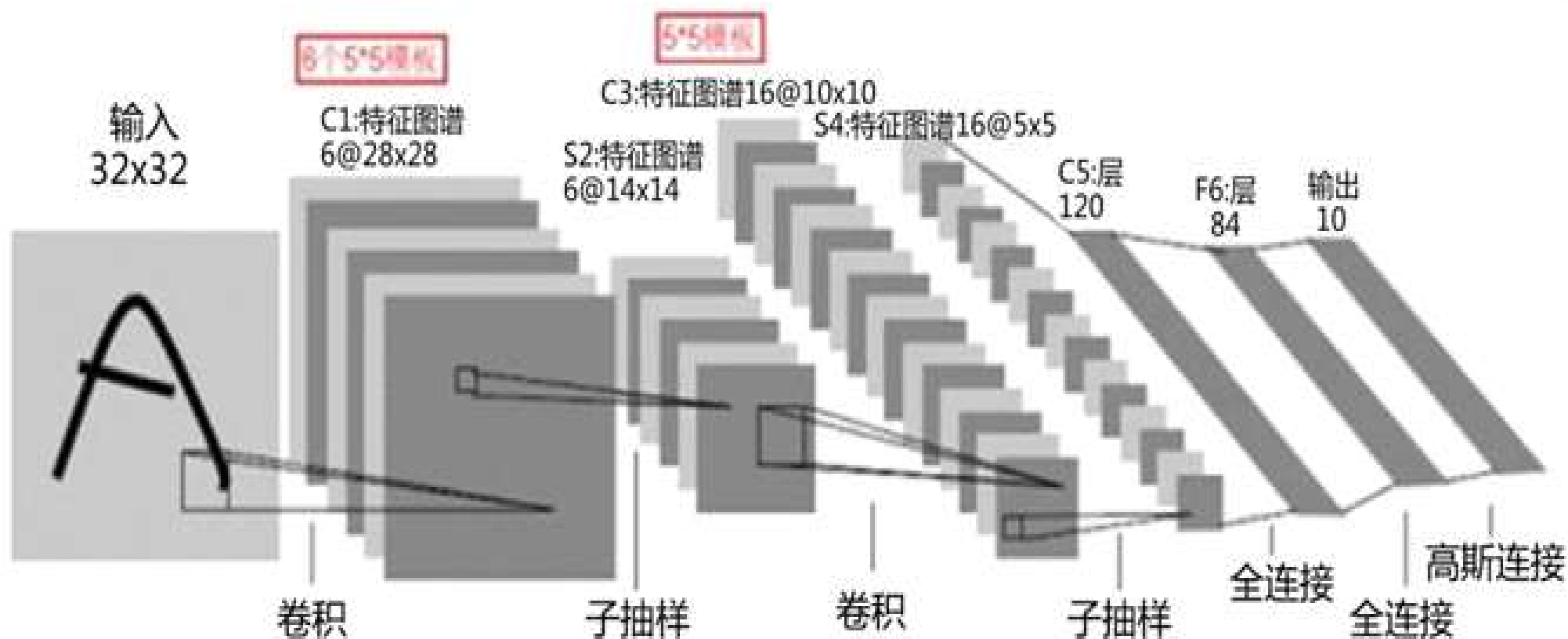
8.6.4 卷积神经网络的应用

- 一种典型的用来识别数字的卷积网络是[LeNet-5](#)。美国大多数银行当年用它识别支票上面的手写数字，达到了商用地步，说明该算法具有很高的准确性。



8.6.4 卷积神经网络的应用

- **LeNet-5**是一个数字手写系统，不包含输入层，共有**7**层，每层都包含可训练参数（连接权重）。输入图像为**32*32**大小。



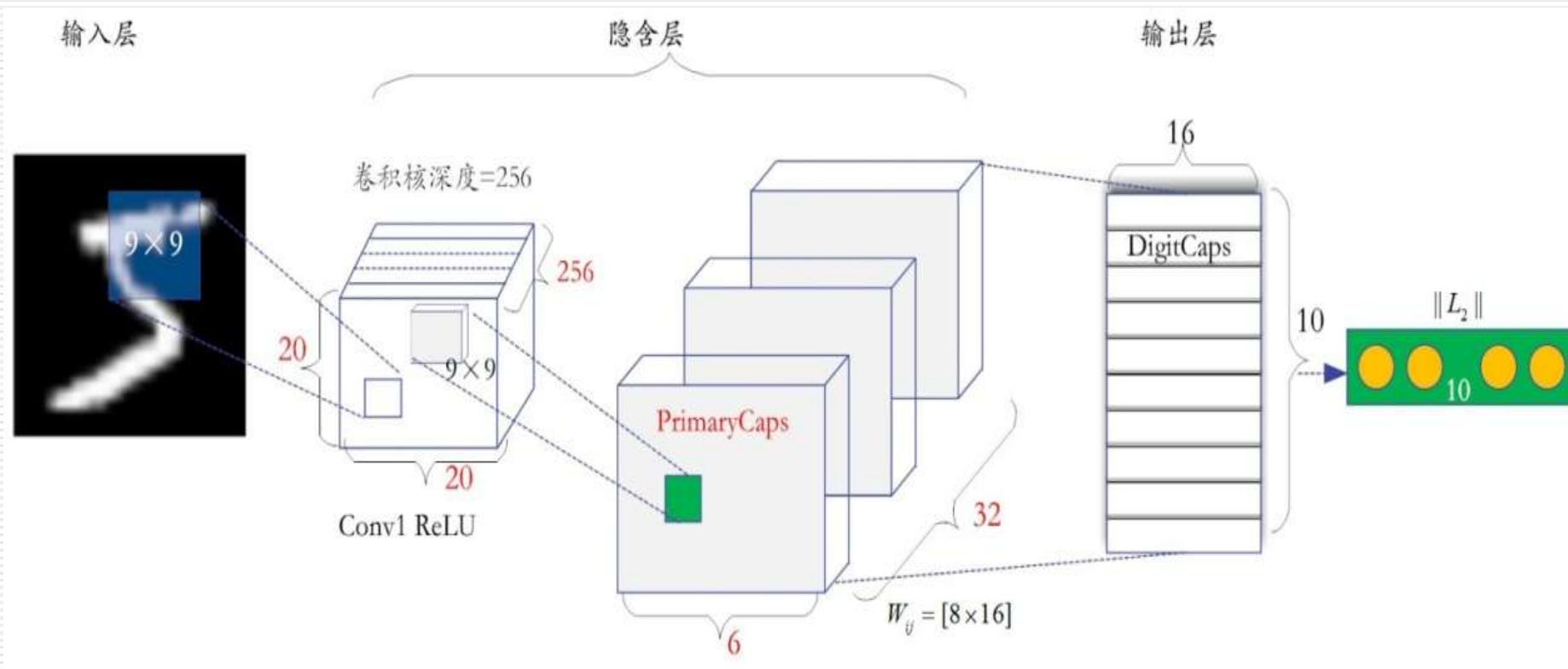
8.6.5 胶囊网络

- ❑ 针对卷积神经网络训练数据需求大、环境适应能力弱、可解释性差、数据分享难等不足，**2017年10月，Geoffrey E. Hinton**教授等在“神经信息处理系统大会上发表论文，提出了新型神经网络结构——**胶囊网络（Capsule Networks）**。
- ❑ 胶囊是一个包含多个神经元的载体，每个神经元表示了图像中出现的特定实体的各种属性。
- ❑ 胶囊不是传统神经网络中的一个神经元，而是一组神经元。
- ❑ **胶囊网络的核心思想**：胶囊里封装的检测特征的相关信息是以向量的形式存在的，胶囊的输入是一个向量，是用一组神经元来表示多个特征。

8.6.5 胶囊网络

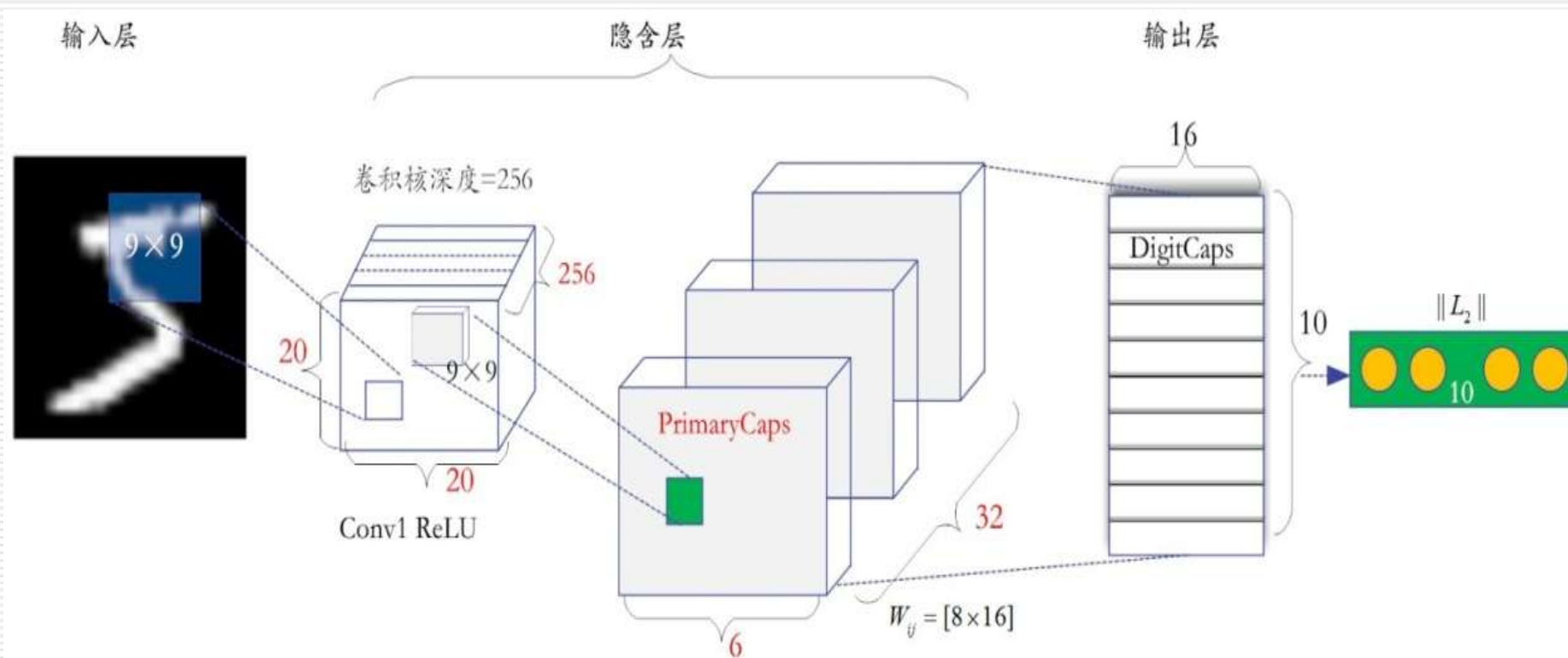
❑ **输入层**：数字图片经过标准的卷积层，有256个通道，每个通道均用 9×9 的卷积核，将输入层图片中的像素亮度转化成局部特征输出，作为Conv1层的输入。

❑ **PrimaryCaps层**：卷积的胶囊层，包含32个胶囊。PrimaryCaps才是胶囊真正开始的地方。



8.6.5 胶囊网络

- ❑ **DigitCaps层**：胶囊网络的全连接层。
- ❑ 要识别的是**10**类数字（**0~9**），因此该层的胶囊个数共有**10**个，每个胶囊表示的向量中元素的个数为**16**，代表着不同状态下的同一个数字。



8.6.5 胶囊网络

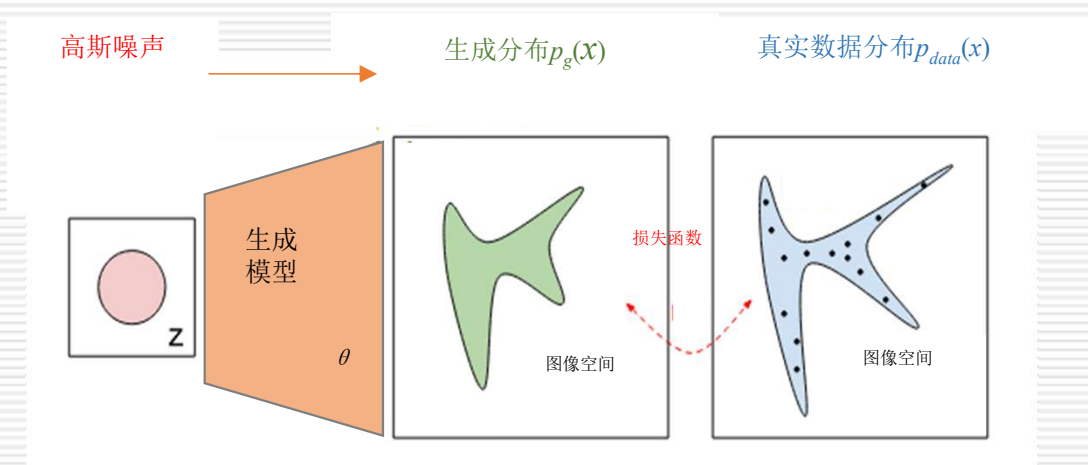
- 使用胶囊神经网络需要的训练数据量，远小于卷积神经网络，它采用动态路由协议算法，仅使用三层网络便可表现出很出色的性能，能够与深度卷积神经网络相当。
- 胶囊网络解决了卷积神经网络存在的信息丢失、视角变化等问题。
- 由于胶囊网络具有分别处理不同属性的能力，相比于**CNN**可以提高对图像变换的鲁棒性，在图像分割中也会有出色的表现。
- 胶囊网络相对于卷积网络的工作机理更接近人脑的工作方式。

第8章 人工神经网络及其应用

- 8.1 神经元与神经网络
- 8.2 BP神经网络及其学习算法
- 8.3 BP神经网络在模式识别中的应用
- 8.4 Hopfield神经网络及其改进
- 8.5 Hopfield神经网络的应用
- 8.6 卷积神经网络与深度学习
- 8.7 生成对抗网络及其应用

8.7.1 生成对抗网络的基本原理

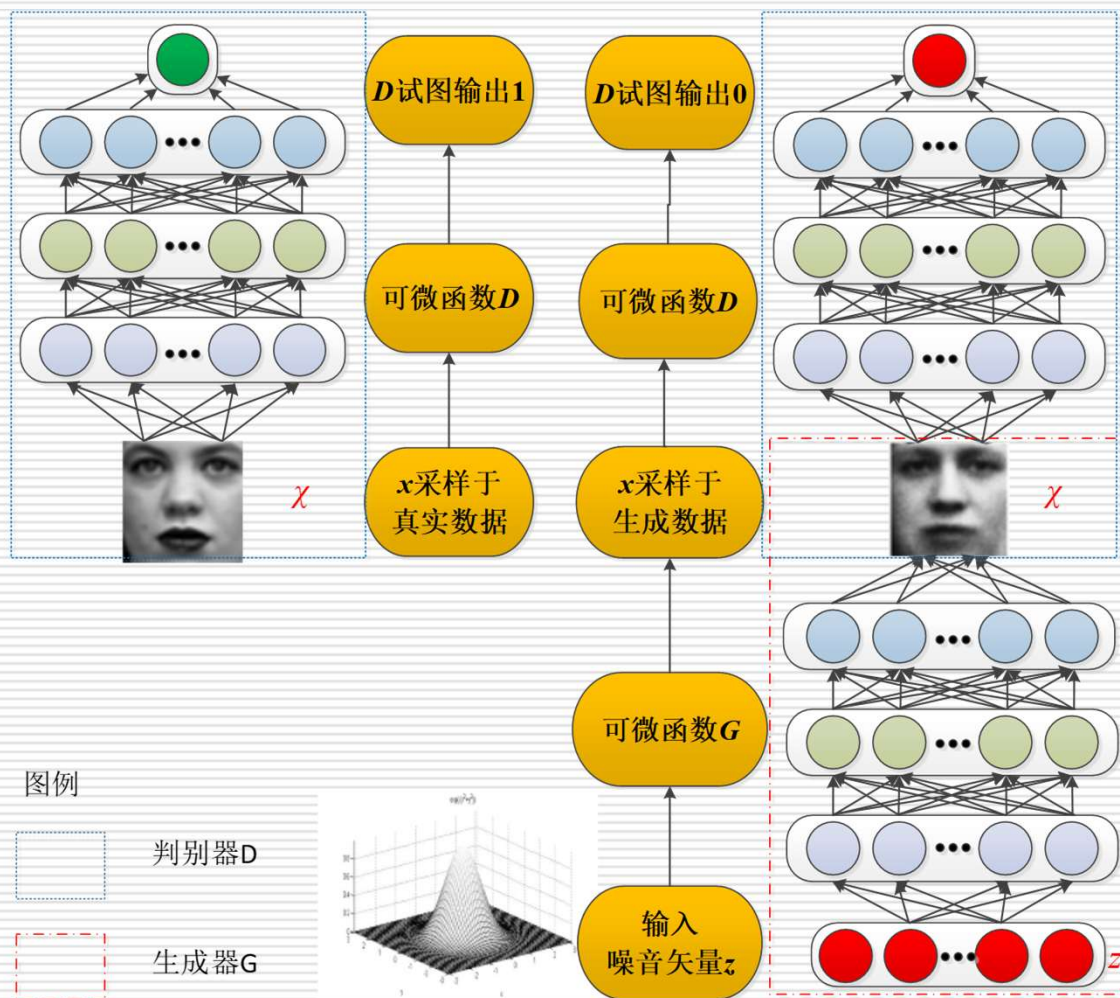
- 深度学习的模型可大致分为判别式模型和生成式模型
- 目前深度学习取得的成果主要集中在判别式模型
- 生成式模型是一个极具挑战的机器学习问题：首先，对真实世界进行建模需要大量先验知识，建模的好坏直接影响生成式模型的性能；其次，真实世界的的数据往往非常复杂，拟合模型所需计算量往往非常庞大，甚至难以承受。
- **Goodfellow**等于**2014**年提出一种新型生成式模型——生成对抗网络（**GAN, generative adversarial network**），使用对抗训练机制对两个神经网络进行训练



8.7.1 生成对抗网络的基本原理

- 深度学习模型分为判别式模型和生成式模型。
- 判别模型是将一个高维的感官输入映射为一个类别标签。
- 生成网络把随机点变成与数据集相似的图片。
- 目前深度学习取得的成果主要集中在判别式模型。
- **Goodfellow**等于**2014**年提出生成对抗网络（**GAN**），使用对抗训练机制对两个神经网络进行训练。

8.7.2 生成对抗网络的结构与训练



金庸

《射雕英雄传》：
老顽童周伯通被
困桃花岛创造的
“左右互搏”之
术。

8.7.2 生成对抗网络的结构与训练

假币制作与识别：



8.7.2 生成对抗网络的结构与训练

GAN的两个相互交替的训练阶段：

- 固定生成网络，训练判别网络
- 固定判别网络，训练生成网络

两个网络相互对抗的过程，就是各自网络参数不断调整的过程，即学习过程。

GAN极大地提高图像生成质量，进一步推动了计算机视觉领域的发展。

8.7.2 生成对抗网络的结构与训练

GAN在训练中容易出现一些问题，训练过程具有强烈的不稳定性，实验结果随机，具体表现：

- 训练过程难以收敛，经常出现震荡；
- 训练收敛，但是出现模式崩溃(model collapse)。
- 训练收敛，但是**GAN**还会生成一些没有意义或者现实中不可能出现的图片。

生成对抗网络的应用

- 深度学习在博弈中的应用
- 深度学习在医学影像识别中的应用
- 生成对抗网络在图像处理中的应用
- 生成对抗网络在语言处理中的应用

生成对抗网络在图像处理中的应用

图像修复



生成对抗网络在图像处理中的应用

图像风格迁移



生成对抗网络在图像处理中的应用

图像风格迁移



生成对抗网络在图像处理中的应用

图像风格迁移



生成对抗网络在图像处理中的应用

图像风格迁移



德国小镇



另一种风格的《沉船》

19世纪英国画家特纳的《沉船》

生成对抗网络在图像处理中的应用

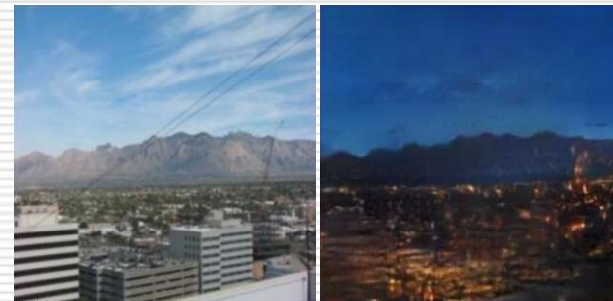
图像翻译



地图生成航拍图像



轮廓图像生成照片



白天图像生成对应夜景

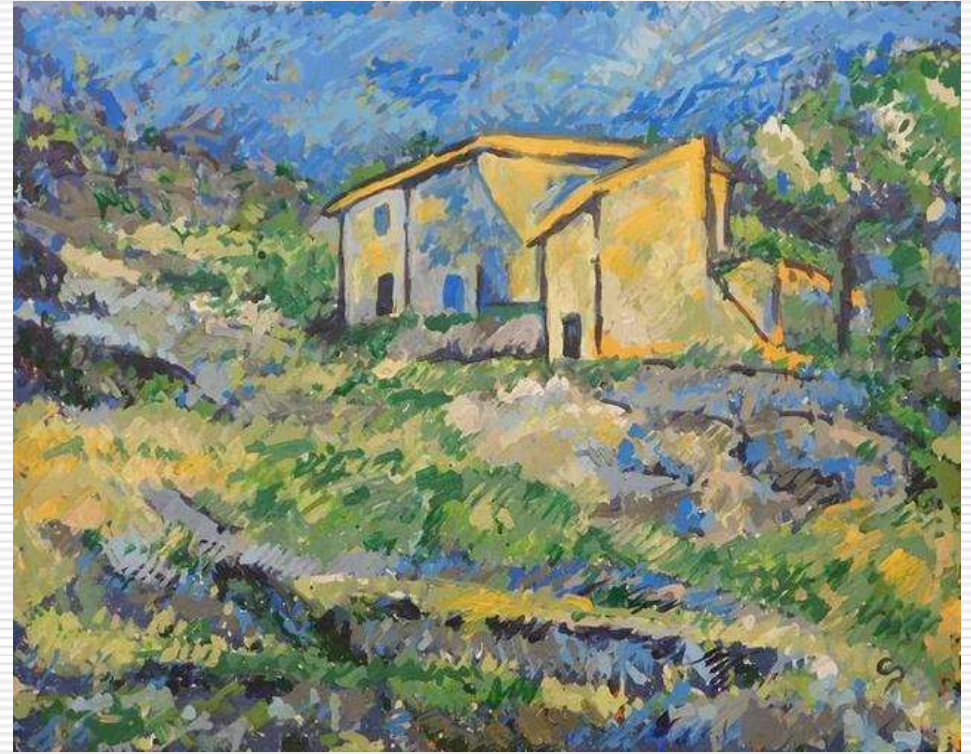


多模态图像翻译

生成对抗网络在艺术创作中的应用



Deep Dream的画作

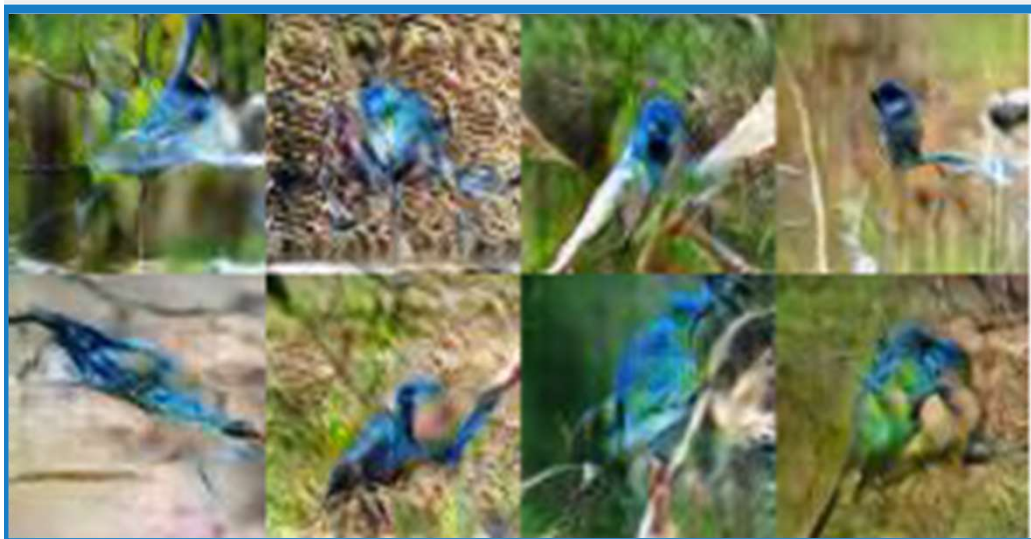


机器人作画

生成对抗网络在语言处理中的应用

从文字描述生成图片

这只小鸟有着小小的鸟喙、胫骨和双足，蓝色的冠部和覆羽，以及黑色的脸颊。



这朵花有着长长的粉色花瓣和朝上的橘黄色雄蕊。



生成对抗网络在语言处理中的应用

诗歌写作

九歌

2017年清华孙茂松团队的计算机古诗作诗系统检验现有神经网络深度学习主流模型的能力

生成对抗网络在视频生成中的应用

AI换脸术：

- 换脸：以**GAN**为代表的**AI**换脸术
- 南加州大学的**Pinscreen**团队的实时**3D**变脸技术。
- 美国国防部**AI**侦测工具，以**AI**攻**AI**，反换脸精度**99%**。

生成对抗网络在视频生成中的应用

新闻播报机器人

- 2018年11月7-9日，在乌镇第五届世界互联网大会上，新华社对外宣布：中国首个“人工智能主持人”正式上岗。



生成对抗网络在视频生成中的应用

2019.2.19新华社发布站立式合成主播



邱浩——新小浩

屈萌——新小萌



THE END